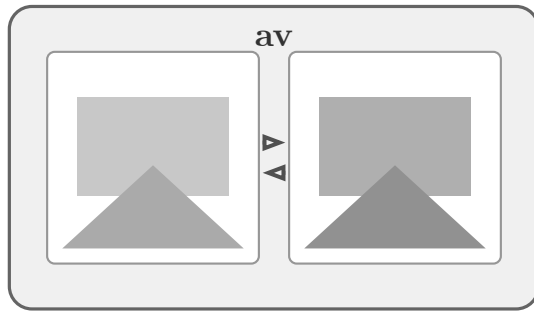




Advanced Pixel Lens

구현 가이드

Implementation Guide

Version 0.2 2026-02-28

크로스 플랫폼 GPU 가속 이미지 뷰어 / 비교 도구

차례

1. 개요	1
1.1. 프로젝트 소개	1
1.2. 기술 스택	2
1.3. 아키텍처 개관	2
2. 빌드 시스템	4
2.1. CMake 설정	4
2.2. 의존성 목록	5
2.3. 플랫폼별 링킹	5
2.4. 빌드 명령	5
2.4.1. Release 빌드	5
2.4.2. Debug 빌드	5
2.4.3. macOS (Apple Silicon)	6
2.4.4. Windows (Visual Studio 2022 + ClangCL)	6
3. 핵심 데이터 구조	7
3.1. ImageEntry	7
3.2. ViewportState	8
3.3. ChannelMode 및 DiffState	8
3.4. CliOptions	9
3.5. AppState	10
3.6. 다이얼로그 관련 구조체	13
3.6.1. SaveItemState	13
3.6.2. ImageSaveDialog	13
3.6.3. ChartSaveDialog	13
3.6.4. StatsSaveDialog	14
3.6.5. OpenFileDialogState	14
3.7. Phase 5 상태 구조체	15
3.7.1. RoiState	15
3.7.2. SequenceState	15
3.7.3. OverlayState	15
3.8. 데이터 흐름	16
4. 렌더링 파이프라인	17
4.1. OpenGL 초기화	17
4.1.1. SDL3 컨텍스트 생성	17
4.1.2. glad2 함수 로딩	17

4.1.3. Dear ImGui 초기화	17
4.2. 셰이더 구성	17
4.2.1. Vertex Shader (공통)	18
4.2.2. Image Fragment Shader	18
4.2.3. Diff Fragment Shader	19
4.2.4. SSIM Heatmap Shader	19
4.3. 뷰포트 변환 수학	20
4.3.1. 순방향 변환 (Shader)	20
4.3.2. 역방향 변환 (CPU)	20
4.3.3. 좌표계 정의	20
4.4. FBO (Framebuffer Object)	20
4.4.1. FBO 구조체	21
4.4.2. 렌더 흐름	21
4.5. ScreenQuad	21
4.6. 텍스처 관리	22
4.6.1. 업로드 함수	22
4.6.2. 필터링 전략	22
4.7. 전체 렌더링 흐름	22
4.7.1. 단일 이미지 렌더링 절차	23
4.7.2. Diff 렌더링 절차	23
5. 이미지 처리	25
5.1. stb_image 로딩	25
5.1.1. ImageEntry 구조체	25
5.1.2. 지원 포맷	26
5.1.3. 로딩 흐름	26
5.2. HDR 지원	27
5.2.1. GPU 업로드 헬퍼	27
5.3. LRU 캐시	27
5.3.1. LRU 동작 시나리오	28
5.3.2. 캐시 도식	28
5.4. 메모리 관리	28
5.5. 이미지 회전	29
5.5.1. 인터페이스	29
5.5.2. 변환 공식	29
5.5.3. 구현 세부	29
6. 뷰포트 시스템	31
6.1. 줌 레벨 시스템	31

6.1.1. 줌 탐색	31
6.1.2. ViewportState 구조체	32
6.2. 패닝	32
6.3. Fit-to-Window	32
6.4. 클램핑	33
6.5. 뷰포트 동기화	33
7. 비교 엔진	34
7.1. GPU Diff 렌더러	34
7.2. Diff 모드 3종 상세	34
7.2.1. PixelAbsolute (u_diff_mode = 0)	34
7.2.2. PixelRelative (u_diff_mode = 1)	35
7.2.3. FalseColor (u_diff_mode = 2)	35
7.2.4. Diff 모드 비교 요약	36
7.3. CPU SSIM 비동기 계산	36
7.3.1. SSIMComputer 클래스	36
7.3.2. SSIMResult 구조체	36
7.3.3. SSIM 공식	37
7.3.4. SSIM 상수	37
7.3.5. 비동기 계산 과정	37
7.4. 채널 분리	37
7.5. 비교 모드 시각화	38
7.5.1. 픽셀 그리드 오버레이	38
8. 사용자 인터페이스	39
8.1. 패널 레이아웃	39
8.2. 메뉴바	40
8.3. 상태바	41
8.4. 이미지 정보 팝업	41
8.5. 줌 HUD	41
8.6. 경계 렌더링 (Border)	42
8.7. Pathfinder (미니맵)	42
8.8. 픽셀 값 표시	43
8.9. 픽셀 값 풍선말 (Pixel Balloon)	44
8.10. 드래그 줌 (Drag-to-Zoom)	44
8.11. Statistics 창 (Ctrl+S)	45
8.12. 차트/통계 창 상호 배타적 토크	47
8.13. Histogram 창 (Ctrl+H)	47
8.14. H-Line Cut 창 (Ctrl+L)	48

8.15. V-Line Cut 창 (Ctrl+Y)	49
8.16. 파일 열기 다이얼로그	49
8.16.1. Open Images 창 (Shift+Cmd+O)	49
8.16.2. SDL 파일 다이얼로그 필터	50
8.16.3. 비동기 처리 흐름	50
8.17. Save 다이얼로그 시스템	50
8.17.1. Image Save 다이얼로그	50
8.17.2. Chart Save 다이얼로그	50
8.17.3. Stats Save 다이얼로그	51
8.18. 이미지 회전 UI	51
8.19. Chart Export 렌더링 파이프라인	51
8.19.1. SoftRenderer	51
8.19.2. Histogram PNG 내보내기	52
8.19.3. Line Cut PNG 내보내기	52
8.19.4. CSV 내보내기	53
9. 사용법	54
9.1. CLI 옵션	54
9.2. 키보드 단축키	55
9.2.1. 줌 조작	55
9.2.2. 패닝 / 내비게이션	55
9.2.3. 비교 모드 전환	56
9.2.4. Diff 파라미터	56
9.2.5. 채널 분리	56
9.2.6. UI 토클	57
9.2.7. 시퀀스 탐색	58
9.3. 마우스 조작	58
9.4. 사용 예시	58
9.5. 워크플로우 시나리오	59
9.5.1. 시나리오 1: 렌더링 회귀 테스트	59
9.5.2. 시나리오 2: 색 보정 검토	59
9.5.3. 시나리오 3: 머신러닝 출력 분석	59
9.5.4. 시나리오 4: ROI 영역 집중 분석	59
9.5.5. 시나리오 5: 비디오 프레임 시퀀스 비교	59
9.5.6. 시나리오 6: Scatter Plot 색 분포 분석	59
10. 앱 아이콘	61
10.1. 프로시저럴 생성	61
10.2. 128×128 디자인 구조	61

10.2.1. 3×3 컬러 그리드	61
10.2.2. 돋보기 오버레이	62
11. Phase 5: 분석 기능 확장	63
11.1. ROI (Region of Interest) 선택 + 영역 통계	63
11.1.1. ROI 드래그 선택 흐름	63
11.1.2. ROI 오버레이 렌더링	63
11.1.3. compute_roi_stats	63
11.1.4. 상태바 표시	64
11.2. 이미지 시퀀스 탐색	64
11.2.1. scan_image_directory	64
11.2.2. 탐색 키 처리	64
11.2.3. 상태바 시퀀스 표시	65
11.3. Overlay/Blend 비교 모드	65
11.3.1. Blend 모드	65
11.3.2. Curtain 모드	65
11.3.3. 레이아웃 전환	66
11.3.4. 상태바 표시	66
11.4. Scatter Plot	66
11.4.1. extract_scatter_plot	66
11.4.2. 시각화	66
11.5. Linux AppImage 아키텍처 자동 감지	67
12. Phase 6: v0.11 기능 확장	68
12.1. Crosshair Overlay	68
12.1.1. 상태 필드	68
12.1.2. 렌더링 방식	68
12.1.3. 키 바인딩	68
12.2. PSNR 자동 계산	68
12.2.1. 구현	68
12.2.2. 캐시 무효화	68
12.3. Tolerance Diff	69
12.3.1. 상태 필드	69
12.3.2. GPU/CPU 이중 경로	69
12.3.3. 키 바인딩	69
12.4. Slideshow 자동 재생	69
12.4.1. SlideshowState	69
12.4.2. DeltaTime 기반 카운트다운	70
12.4.3. 키 바인딩	70

12.5. Histogram Compare	70
12.5.1. compare 모드 체크박스	70
12.5.2. 이중 히스토그램 오버레이	70
12.6. Dynamic Window Title	70
12.6.1. 타이틀 포맷	71
12.6.2. 구현	71
12.7. Border Toggle, Pixel Format 및 av.ini 설정 영속화	71
12.7.1. 토글 메커니즘	71
12.7.2. av.ini 영속화	71

1. 개요

1.1. 프로젝트 소개

av(Advanced Pixel Lens)는 렌더링 엔지니어, 테크니컬 아티스트, 연구자를 위한 GPU 가속 이미지 비교 도구이다. 두 장의 이미지를 나란히 배치하여 픽셀 단위의 절대/상대 차이, False-Color 히트맵, 그리고 SSIM(Structural Similarity Index) 분석을 실시간으로 수행할 수 있다.

프로젝트의 핵심 설계 목표는 다음과 같다.

- **실시간 비교:** 모든 diff 연산을 GLSL fragment shader에서 수행하여 고해상도 이미지에서도 즉각적인 시각적 피드백을 제공한다.
- **HDR 지원:** 8비트 LDR(RGBA8)과 32비트 HDR(RGBA32F) 이미지를 모두 지원한다.
- **색상 관리:** lcms2를 통한 ICC 프로파일 기반 색상 변환을 제공한다.
- **경량 바이너리:** stb_image 등 header-only 라이브러리와 FetchContent 기반 빌드로 외부 의존성을 최소화한다.
- **키보드 중심 워크플로우:** vi 스타일의 hjkl 탐색과 단축키 기반 조작을 지원한다.

1.2. 기술 스택

항목	선택	이유
GUI 프레임워크	SDL3 + Dear ImGui (docking branch)	네이티브 윈도우 관리, 도킹 가능 패널, 파일 다이얼로그 내장
이미지 로딩	stb_image	Header-only, LDR/HDR 지원, 빌드 의존성 제로
렌더링 백엔드	OpenGL 3.3 Core	macOS/Linux/Windows 공통, 충분한 셰이더 기능
GL 로더	glad2	OpenGL 3.3 Core 전용 로더 자동 생성
색상 관리	lcms2	ICC v4 프로파일 변환, 시스템 설치 활용
빌드 시스템	CMake 3.24+	FetchContent로 소스 의존성 자동 다운로드
C++ 표준	C++20	<code>std::jthread</code> , concepts 등 현대적 기능 활용
GPU diff	GLSL fragment shader	실시간 픽셀 비교 (Absolute, Relative, FalseColor)
CPU diff	커스텀 SSIM	<code>std::jthread</code> 비동기 연산, 11×11 Gaussian 커널

표 1: av 기술 스택 요약

1.3. 아키텍처 개관

av의 아키텍처는 세 개의 수평 계층으로 구성된다. UI Layer는 사용자 인터랙션과 화면 표시를 담당하고, Engine Layer는 이미지 로딩·뷰포트·diff 연산 등 핵심 로직을 처리하며, Platform Layer는 운영체제 및 그래픽 API와의 인터페이스를 제공한다.

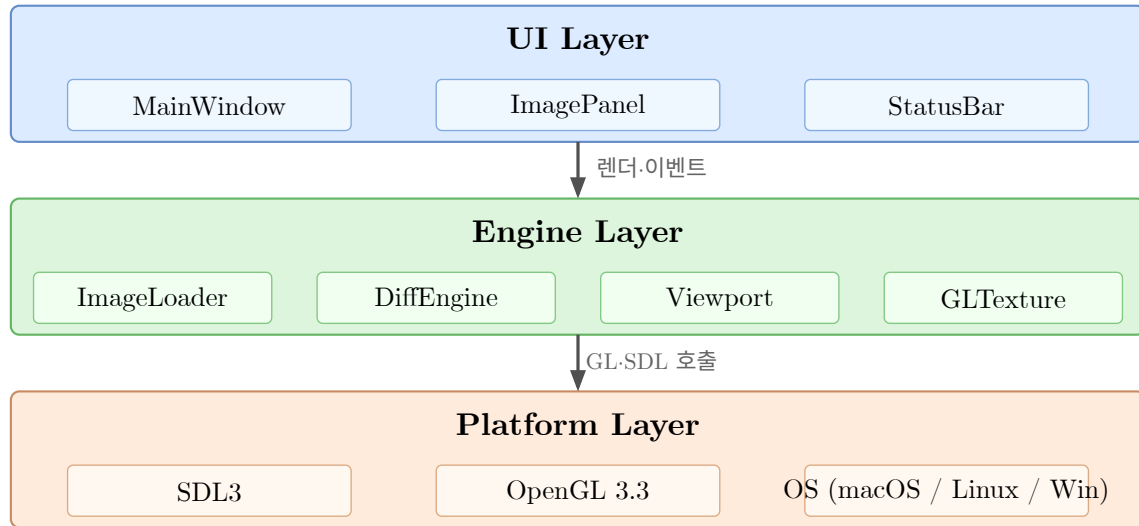


그림 1: av 3계층 아키텍처 다이어그램

2. 빌드 시스템

2.1. CMake 설정

av는 CMake 3.24 이상을 요구하며, `FetchContent` 를 통해 대부분의 외부 라이브러리를 소스에서 자동으로 가져온다. C++ 표준은 20으로 설정되어 `std::jthread`, structured bindings, concepts 등을 활용한다.

```
cmake_minimum_required(VERSION 3.24)
project(av CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

include(FetchContent)

FetchContent_Declare(SDL3
  GIT_REPOSITORY https://github.com/libsdl-org/SDL.git
  GIT_TAG release-3.2.0)

FetchContent_Declare(imgui
  GIT_REPOSITORY https://github.com/ocornut/imgui.git
  GIT_TAG docking)

FetchContent_Declare(glad
  GIT_REPOSITORY https://github.com/Dav1dde/glad.git
  GIT_TAG glad2)

FetchContent_MakeAvailable(SDL3 imgui glad)
```

`stb_image`는 `header-only` 라이브러리로, 별도의 `stb_impl.cpp`에서 `#define STB_IMAGE_IMPLEMENTATION`을 선언하여 구현체를 포함시킨다. `lcms2`는 시스템에 설치된 패키지를 `find_package` 또는 `pkg-config`를 통해 연결한다.

2.2. 의존성 목록

라이브러리	Git Tag / 버전	용도
SDL3	release-{}3.2.0	윈도우 생성, 이벤트 루프, 파일 다이얼로그
Dear ImGui	docking	UI 프레임워크 (도킹 레이아웃)
glad2	glad2	OpenGL 3.3 Core 함수 포인터 로더
stb	master	이미지 로딩 (stb_image.h) 및 저장
lcms2	시스템 설치	ICC 프로파일 기반 색상 관리

표 2: FetchContent 및 시스템 의존성

2.3. 플랫폼별 링킹

OpenGL 프레임워크는 운영체제마다 링크 대상이 다르다.

플랫폼	링크 타겟	비고
macOS	OpenGL.framework	Deployment target 12.0, arm64
Linux	libGL, libdl	Mesa 또는 NVIDIA 드라이버
Windows	opengl32.lib	MSVC / clang-cl (ClangCL), STBI_WINDOWS_UTF8 경로 지원

표 3: 플랫폼별 OpenGL 링킹

macOS에서는 CMAKE_OSX_DEPLOYMENT_TARGET 을 12.0으로, 아키텍처를 arm64 로 설정한다. macOS/Linux에서는 ~/.local/bin/av 에, Windows에서는 %LOCALAPPDATA%\av\ 에 자동 복사된다.

2.4. 빌드 명령

2.4.1. Release 빌드

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)
```

2.4.2. Debug 빌드

```
cmake -B build-debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build-debug
```

2.4.3. macOS (Apple Silicon)

```
cmake -B build \  
  -DCMAKE_BUILD_TYPE=Release \  
  -DCMAKE_OSX_ARCHITECTURES=arm64 \  
  -DCMAKE_OSX_DEPLOYMENT_TARGET=12.0  
cmake --build build -j$(sysctl -n hw.ncpu)
```

빌드가 완료되면 `build/av` 실행 파일이 생성되며, 포스트 빌드 스크립트에 의해 `~/local/bin/av`로 자동 복사된다.

2.4.4. Windows (Visual Studio 2022 + ClangCL)

```
cmake -B build -G "Visual Studio 17 2022" -T ClangCL  
cmake --build build --config Release -j %NUMBER_OF_PROCESSORS%
```

빌드 스크립트 `script\build-windows.bat` 사용 가능.

3. 핵심 데이터 구조

av의 전체 상태는 `AppState` 구조체 하나에 집약된다. 이 장에서는 `app.h`에 정의된 핵심 데이터 구조를 하나씩 살펴본다.

3.1. ImageEntry

`ImageEntry`는 로드된 이미지 한 장의 모든 정보를 담는 구조체이다.

```
struct ImageEntry {
    std::string      path;
    std::vector<uint8_t> pixels;      // CPU-side RGBA8
    std::vector<float> pixels_f32;    // CPU-side RGBA32F (HDR)
    std::vector<uint16_t> pixels_orig; // PPM 원본 픽셀값 (P2/P3/P5/P6)
    int   width      = 0;
    int   height     = 0;
    int   channels    = 0;
    int   ppm_maxval = 0;              // PPM maxval (0이면 PPM 아님)
    unsigned int texture_id = 0;      // OpenGL texture (GLuint)
    bool  loaded     = false;
    bool  is_hdr     = false;
};
```

필드	타입	설명
<code>path</code>	<code>std::string</code>	이미지 파일의 절대 경로
<code>pixels</code>	<code>vector<uint8_t></code>	CPU측 RGBA8 픽셀 데이터 (LDR)
<code>pixels_f32</code>	<code>vector<float></code>	CPU측 RGBA32F 픽셀 데이터 (HDR)
<code>pixels_orig</code>	<code>vector<uint16_t></code>	PPM 원본 픽셀값 (P2/P3/P5/P6, 0~maxval)
<code>width</code>	<code>int</code>	이미지 너비 (픽셀)
<code>height</code>	<code>int</code>	이미지 높이 (픽셀)
<code>channels</code>	<code>int</code>	원본 채널 수 (1~4)
<code>ppm_maxval</code>	<code>int</code>	PPM maxval 값 (0이면 PPM 아님)
<code>texture_id</code>	<code>unsigned int</code>	OpenGL 텍스처 핸들 (GLuint)
<code>loaded</code>	<code>bool</code>	로딩 완료 여부
<code>is_hdr</code>	<code>bool</code>	HDR(32bit float) 이미지 여부

표 4: ImageEntry 필드 설명

LDR 이미지는 `pixels` 에, HDR 이미지(`.hdr`, `.exr`)는 `pixels_f32` 에 저장된다. GPU 업로드 후에도 CPU측 데이터는 유지되어 SSIM 연산 등에 사용된다.

3.2. ViewportState

`ViewportState` 는 패널 하나의 줌·팬 상태를 나타낸다.

```
struct ViewportState {
    float zoom = 1.0f; // 표시 배율: 0.1 ~ 256.0
    float pan_x = 0.0f; // 중심으로부터의 x 오프셋 (image-pixel)
    float pan_y = 0.0f; // 중심으로부터의 y 오프셋 (image-pixel)
    bool fit = true; // fit-to-window 모드
};
```

필드	타입	설명
<code>zoom</code>	<code>float</code>	표시 배율. 0.125x~256x 범위의 이산 단계
<code>pan_x</code>	<code>float</code>	이미지 중심에서 오른쪽(+) 방향 오프셋 (image-{ pixel 단위)
<code>pan_y</code>	<code>float</code>	이미지 중심에서 아래쪽(+) 방향 오프셋 (image-{ pixel 단위)
<code>fit</code>	<code>bool</code>	<code>true</code> 이면 이미지를 패널 크기에 맞춰 자동 스케일링

표 5: ViewportState 필드 설명

줌은 다음 12단계 중 하나의 값을 가진다:

```
0.125 0.25 0.5 1.0 2.0 4.0 8.0 16.0 32.0 64.0 128.0 256.0
```

`fit = true` 일 때 줌 값은 `min(view_w / img_w, view_h / img_h)` 로 자동 계산되며, 팬은 `(0, 0)` 으로 리셋된다.

3.3. ChannelMode 및 DiffState

```
enum class ChannelMode { RGB = 0, Red = 1, Green = 2, Blue = 3 };
```

`ChannelMode` 는 현재 표시 중인 채널을 결정한다. R, G, B 키로 전환하며 개별 채널을 그레이스케일로 시각화한다.

```
enum class PixelFormat { Decimal = 0, Hex0x = 1, HexH = 2 };
```

PixelFormat 은 정수 픽셀값의 표시 형식을 결정한다. Ctrl+X 로 순환 전환하며, Decimal (128), Hex0x (0x80), HexH (80h) 세 가지 형식을 지원한다. HDR float 값은 형식 토글 대상이 아니다.

```
struct DiffState {
    enum class Mode {
        None, PixelAbsolute, PixelRelative, FalseColor, SSIM
    };
    Mode mode = Mode::None;
    float amplify = 1.0f;
    float ssim_score = -1.0f;
    unsigned int ssim_texture_id = 0;
    bool ssim_computing = false;
};
```

필드	타입	설명
mode	Mode	현재 diff 모드 (None, PixelAbsolute, PixelRelative, FalseColor, SSIM)
amplify	float	diff 증폭 계수 (↖ / ↗ 키로 조절, 기본 1.0)
ssim_score	float	마지막 SSIM 연산 결과 (-1.0 = 미계산)
ssim_texture_id	unsigned int	SSIM 히트맵 텍스처 핸들
ssim_computing	bool	SSIM 비동기 연산 진행 중 여부

표 6: DiffState 필드 설명

Diff 모드별 동작은 Section 4.2.3 에서 상세히 다룬다.

3.4. CliOptions

```
struct CliOptions {
    std::string image_a;
    std::string image_b;
    DiffState::Mode diff_mode = DiffState::Mode::None;
    float zoom = 0.0f; // 0 = fit
    bool sync = true;
    float amplify = 1.0f;
    bool fullscreen = false;
    int win_w = 1280;
    int win_h = 720;
    std::string icc_profile;
    bool no_color_mgmt = false;
    bool windowed = false;
};
```



```

bool        no_border    = false;
int         pan_step     = 32;
std::array<uint32_t, 3> border_colors =
    {0xE6FF00FFu, 0xE600FFFFu, 0xE6FFFF00u};
};

```

`CliOptions` 는 커맨드라인 인자를 파싱한 결과를 보관하며, `AppState` 초기화 시 각 필드에 복사된다. `border_colors` 배열은 A 패널(마젠타), B 패널(시안), Diff 패널(옐로)의 테두리 색상을 ARGB 형식으로 저장한다. `no_border` 는 `-nb` 옵션 사용 시 테두리 숨김 상태로 시작한다.

3.5. AppState

```

struct AppState {
    std::array<ImageEntry, 2>    images;
    std::array<ViewportState, 2> views;
    DiffState diff;
    bool  sync_viewports  = true;
    bool  show_ui         = false;
    bool  show_histogram  = false;
    bool  show_hline_cut  = false;
    bool  show_vline_cut  = false;
    bool  show_stats      = false;
    bool  show_info       = false;
    bool  show_pixel_info = false;
    int   pathfinder_mode = 1;      // 0=hidden, 1=image(P), 2=schematic(Ctrl+P)
    bool  swap_images     = false;
    int   active_panel    = 0;
    bool  quit            = false;
    float zoom_hud_timer  = 0.0f;
    float last_zoom_a     = 1.0f;
    float last_zoom_b     = 1.0f;
    ChannelMode channel_mode = ChannelMode::RGB;
    int   pan_step        = 32;
    std::array<uint32_t, 3> border_colors;
    bool  show_borders    = true;   // B key: toggle panel borders
    PixelFormat pixel_format = PixelFormat::Decimal; // Ctrl+X 순환
    ImFont* font_large     = nullptr; // Roboto-Medium 26px
    ImFont* font_medium    = nullptr; // Roboto-Medium 18px
    CliOptions cli;
    bool  show_save_dialog = false;
    ImageSaveDialog image_save;
};

```

```
ChartSaveDialog  chart_save;  
StatsSaveDialog  stats_save;  
OpenDialogState  open_state;  
SDL_Window*      window = nullptr;  
};
```

`AppState` 는 애플리케이션의 모든 런타임 상태를 하나의 구조체에 집약한다. `images[0]` / `images[1]` 은 각각 좌측(A).우측(B) 패널의 이미지이며, `views[0]` / `views[1]` 은 대응하는 뷰포트 상태이다. `sync_viewports` 가 `true` 이면 두 패널의 줌.팬이 동기화된다.

필드	설명
<code>images</code>	A/B 이미지 엔트리 (2개 고정)
<code>views</code>	A/B 뷰포트 상태 (줌, 팬, fit)
<code>diff</code>	Diff 모드 상태
<code>sync_viewports</code>	뷰포트 동기화 여부 (<code>s</code> 키 토글)
<code>show_ui</code>	ImGui 데모 UI 표시 여부 (<code>u</code> 키)
<code>show_histogram</code>	Histogram 창 표시 여부 (<code>Ctrl+H</code>)
<code>show_hline_cut</code>	H-Line Cut 창 표시 여부 (<code>Ctrl+L</code>)
<code>show_vline_cut</code>	V-Line Cut 창 표시 여부 (<code>Ctrl+Y</code>)
<code>show_stats</code>	Statistics 창 표시 여부 (<code>Ctrl+S</code>)
<code>show_pixel_info</code>	커서 위치 픽셀 값 풍선말 표시 (<code>v</code> 키)
<code>pathfinder_mode</code>	미니맵 모드: 0=숨김, 1=이미지(<code>P</code>), 2=배선도(<code>Ctrl+P</code>). 기본 1
<code>swap_images</code>	A/B 이미지 교환 상태 (<code>Shift+Space</code>)
<code>active_panel</code>	현재 포커스된 패널 인덱스 (0 또는 1)
<code>channel_mode</code>	채널 표시 모드 (RGB, R, G, B)
<code>pan_step</code>	<code>Shift+hjkl</code> 팬 이동량 (픽셀, 기본 32)
<code>border_colors</code>	패널 테두리 색상 배열 [A, B, Diff]
<code>show_borders</code>	테두리 표시 여부 (<code>B</code> 키 토글, <code>av.ini</code> 영속화)
<code>pixel_format</code>	픽셀값 표시 형식 (<code>Ctrl+X</code> 순환: Decimal/Hex0x/HexH, <code>av.ini</code> 영속화)
<code>font_medium</code>	Roboto-Medium 18px 폰트 (차트 창 등에서 사용)
<code>show_save_dialog</code>	컨텍스트 인식 Save 다이얼로그 표시 여부 (<code>Shift+Cmd+S</code>)
<code>image_save</code>	<code>ImageSaveDialog</code> 상태 구조체
<code>chart_save</code>	<code>ChartSaveDialog</code> 상태 구조체
<code>stats_save</code>	<code>StatsSaveDialog</code> 상태 구조체
<code>open_state</code>	<code>OpenDialogState</code> 상태 구조체
<code>window</code>	네이티브 파일 다이얼로그용 SDL 윈도우 포인터
<code>roi</code>	<code>RoiState</code> — ROI 선택 모드 상태 (<code>Ctrl+E</code>)
<code>show_roi_stats</code>	ROI 통계 창 표시 여부
<code>sequences[2]</code>	<code>SequenceState[2]</code> — A/B 패널 시퀀스 탐색 상태
<code>overlay</code>	<code>OverlayState</code> — Overlay/Blend 비교 모드 상태 (0)
<code>show_scatter_plot</code>	Scatter Plot 창 표시 여부 (<code>Ctrl+T</code>)

표 7: AppState 주요 필드

3.6. 다이얼로그 관련 구조체

v0.2에서 파일 열기·저장 다이얼로그 시스템을 지원하기 위한 구조체 4종이 추가되었다. v0.3에서는 ROI, 시퀀스 탐색, Overlay, Scatter Plot을 위한 구조체 3종이 추가되었다 (Section 3.7).

3.6.1. SaveItemState

개별 저장 항목(A/B/Diff)의 체크박스 상태와 경로를 보관하는 최소 구조체이다.

```
struct SaveItemState {
    bool checked    = true;
    char path[512]  = {};
};
```

3.6.2. ImageSaveDialog

이미지 저장 다이얼로그의 전체 상태를 보관한다. A/B/Diff 세 이미지를 동시에 저장할 수 있다.

```
struct ImageSaveDialog {
    enum class Format { PNG, BMP, PPM };
    enum class PpmMode { Binary, ASCII };

    Format format      = Format::PNG;
    Format prev_format = Format::PNG; // 확장자 자동 업데이트용
    PpmMode ppm_mode   = PpmMode::Binary;
    int ppm_bits       = 8;          // 8, 10, 12, 16

    SaveItemState items[3];          // 0=A, 1=B, 2=Diff
    bool initialized   = false;
    std::string status_msg;
    bool status_error  = false;
};
```

PPM 모드에서 `ppm_bits` 는 8/10/12/16 비트를 지원하며, `PpmMode::Binary` 는 P6(바이너리), `PpmMode::ASCII` 는 P3(텍스트) 형식으로 저장한다.

3.6.3. ChartSaveDialog

차트 PNG 및 CSV 내보내기 다이얼로그의 전체 상태를 보관한다.

```
struct ChartSaveDialog {
    int export_width    = 1920;
    int export_height   = 1080;
```

```

bool separate_channels = false;

SaveItemState png_items[3]; // PNG 내보내기 (0=A, 1=B, 2=Diff)
SaveItemState csv_items[3]; // CSV 내보내기
bool          initialized      = false;
bool          init_was_histogram = false;
bool          init_was_hcut     = false;
std::string   status_msg;
bool          status_error     = false;
};

```

`init_was_histogram` / `init_was_hcut` 플래그로 활성 창 종류를 추적하여 파일 접미사(`_histogram` / `_hcut` / `_vcut`)를 자동 결정한다. `separate_channels` 가 `true` 이면 R/G/B 채널을 각각 별도 파일 (`_R` / `_G` / `_B`)로 내보낸다.

3.6.4. StatsSaveDialog

Statistics 창의 CSV 저장 다이얼로그 상태를 보관한다.

```

struct StatsSaveDialog {
    SaveItemState items[3]; // 0=A, 1=B, 2=Diff
    bool          initialized = false;
    std::string   status_msg;
    bool          status_error = false;
};

```

3.6.5. OpenFileDialogState

파일 열기 다이얼로그와 “Open Images” 창의 상태를 보관한다.

```

struct OpenFileDialogState {
    bool          show          = false; // Shift+Cmd+O 톤극
    std::string   opened_path; // SDL 콜백에서 설정
    int           open_target   = -1; // 0=A, 1=B
    bool          open_pending  = false; // 메인 루프 처리 대기 중
    bool          clear_other   = false; // 닫을 로드 시 반대 슬롯 클리어
};

```

SDL3의 비동기 파일 다이얼로그 콜백이 `opened_path` 와 `open_pending` 을 설정하면, 메인 루프에서 이를 감지하여 실제 이미지 로딩을 처리한다.

3.7. Phase 5 상태 구조체

v0.3에서 분석 기능 확장을 위한 상태 구조체 3종이 추가되었다.

3.7.1. RoiState

ROI(Region of Interest) 선택 모드의 모든 상태를 보관한다.

```
struct RoiState {
    bool active    = false; // Ctrl+E: ROI 선택 모드 ON/OFF
    bool has_roi   = false; // 유효한 ROI가 선택됨
    int  x = 0, y = 0;      // 이미지 픽셀 좌표 (top-left)
    int  w = 0, h = 0;      // ROI 크기 (픽셀)
    int  panel_idx = 0;      // ROI가 그려진 패널 인덱스

    bool dragging  = false;
    float drag_sx  = 0.0f; // 드래그 시작 화면 좌표
    float drag_sy  = 0.0f;
    int   drag_panel = 0;
};
```

3.7.2. SequenceState

패널별 이미지 시퀀스 탐색 상태를 보관한다. `AppState::sequences[2]` 로 A/B 패널 각각 독립적으로 관리된다.

```
struct SequenceState {
    std::vector<std::string> files; // 디렉토리 내 이미지 목록 (natural sort)
    int                     current_index = -1; // 현재 파일 인덱스 (-1=시퀀스 없음)
};
```

이미지가 로드될 때 `scan_image_directory()` 가 호출되어 동일 디렉토리의 이미지 파일 목록을 natural sort 순서로 채운다. `N` / `Shift+N` 키로 `current_index` 를 증감하면 `open_state` 에 다음 파일 경로가 주입된다.

3.7.3. OverlayState

Overlay/Blend 비교 모드의 상태를 보관한다. `0` 키로 토글되며, 두 패널을 하나의 블렌드 패널로 합성한다.

```
struct OverlayState {
    bool active = false; // 0 키: Overlay 모드 ON/OFF
};
```

```
float alpha    = 0.5f;    // 0=A², 1=B²
enum class Mode { Blend, Curtain } mode = Mode::Blend;
float curtain_x    = 0.5f;    // Curtain 구분선 위치 [0,1]
bool curtain_dragging = false;
};
```

`Mode::Blend` 는 `alpha` 가중치로 A/B를 혼합하고, `Mode::Curtain` 은 화면을 수직으로 나누어 좌측에 A, 우측에 B를 표시한다.

3.8. 데이터 흐름

이미지가 로드되어 화면에 표시되기까지의 데이터 흐름을 그림 2 에 나타낸다. 파일은 `stb_image` 로 디코딩되고, 필요 시 `lcms2` 로 색 변환을 거친 뒤, OpenGL 텍스처로 업로드된다. 셰이더가 뷰포트 변환과 diff 연산을 수행하고, 결과를 FBO에 기록한 뒤 `ImGui::Image` 로 화면에 합성한다.

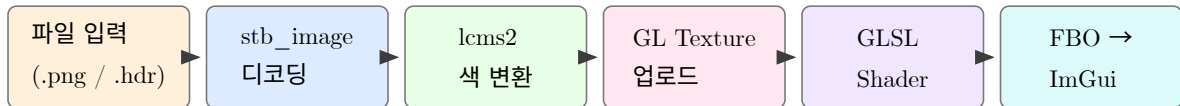


그림 2: 이미지 데이터 흐름: 파일 입력에서 화면 표시까지

4. 렌더링 파이프라인

이 장에서는 av의 GPU 렌더링 파이프라인을 상세히 다룬다. OpenGL 초기화부터 셰이더 4종, 뷰포트 변환 수학, FBO 관리, 그리고 최종 화면 합성까지의 전체 과정을 설명한다.

4.1. OpenGL 초기화

4.1.1. SDL3 컨텍스트 생성

av는 SDL3를 사용하여 OpenGL 3.3 Core Profile 컨텍스트를 생성한다.

```
SDL_GL_SetAttribute(SDL_GL_CONTEXT_MAJOR_VERSION, 3);
SDL_GL_SetAttribute(SDL_GL_CONTEXT_MINOR_VERSION, 3);
SDL_GL_SetAttribute(
    SDL_GL_CONTEXT_PROFILE_MASK,
    SDL_GL_CONTEXT_PROFILE_CORE);

SDL_Window* window = SDL_CreateWindow(
    "av - Advanced Pixel Lens",
    state.cli.win_w, state.cli.win_h,
    SDL_WINDOW_OPENGL | SDL_WINDOW_RESIZABLE);

SDL_GLContext gl_ctx = SDL_GL_CreateContext(window);
```

4.1.2. glad2 함수 로딩

컨텍스트 생성 직후 `gladLoadGL` 로 OpenGL 함수 포인터를 로드한다. 이후 모든 `gl*` 함수 호출이 유효해진다.

4.1.3. Dear ImGui 초기화

Dear ImGui의 SDL3 + OpenGL3 백엔드를 초기화한다. `docking branch`를 사용하므로 `ImGuiConfigFlags_DockingEnable` 플래그를 설정하여 도킹 레이아웃을 활성화한다.

4.2. 셰이더 구성

av는 `shader_sources.h`에 4종의 GLSL 셰이더를 문자열 리터럴로 내장하고 있다. 모든 셰이더는 GLSL 150 (OpenGL 3.2 호환)을 사용한다.

4.2.1. Vertex Shader (공통)

모든 fragment shader가 공유하는 단일 vertex shader이다. full-screen quad의 위치와 UV 좌표를 전달한다.

```
#version 150
in  vec2 a_pos;
in  vec2 a_uv;
out vec2 v_uv;

void main() {
    v_uv = a_uv;
    gl_Position = vec4(a_pos, 0.0, 1.0);
}
```

정점 속성 `a_pos` 는 NDC 좌표 $(-1, -1) \sim (1, 1)$ 범위이며, `a_uv` 는 텍스처 좌표 $(0, 0) \sim (1, 1)$ 범위이다. 6개의 정점(삼각형 2개)으로 화면 전체를 커버하는 quad를 구성한다.

4.2.2. Image Fragment Shader

단일 이미지를 뷰포트 변환과 함께 렌더링하는 셰이더이다.

```
#version 150
uniform sampler2D u_tex;
uniform vec2 u_image_size; // 원본 이미지 크기
uniform vec2 u_view_size; // 뷰포트 크기 (픽셀)
uniform float u_zoom; // 현재 줌 배율
uniform vec2 u_pan; // 팬 오프셋 (image-pixel)
uniform int u_channel; // 0=RGB, 1=R, 2=G, 3=B
out vec4 out_color;

void main() {
    vec2 screen_px = v_uv * u_view_size;
    vec2 img_px = (screen_px - u_view_size * 0.5)
                  / u_zoom
                  - u_pan + u_image_size * 0.5;
    vec2 uv = img_px / u_image_size;

    if (uv.x < 0.0 || uv.x > 1.0 ||
        uv.y < 0.0 || uv.y > 1.0) {
        out_color = vec4(0.15, 0.15, 0.15, 1.0);
        return;
    }
}
```

```

vec4 c = texture(u_tex, uv);
// 채널 선택 (u_channel)
// 픽셀 그라드 (zoom >= 16x)
out_color = c;
}

```

이미지 영역 밖은 어두운 회색(0.15)으로 렌더링된다. `u_channel` uniform으로 개별 채널을 그레이 스케일로 표시할 수 있으며, 줌이 16배 이상일 때 1px 단위의 격자선을 오버레이한다.

4.2.3. Diff Fragment Shader

두 이미지의 차이를 시각화하는 셰이더이다.

```

#version 150
uniform sampler2D u_tex_a;    // 이미지 A
uniform sampler2D u_tex_b;    // 이미지 B
uniform int    u_diff_mode;   // 0=Absolute, 1=Relative, 2=FalseColor
uniform float  u_amplify;     // 증폭 계수

```

Diff 모드별 연산:

- **PixelAbsolute (mode 0):** $|A - B| \times \text{amplify}$ — 채널별 절대 차이를 증폭하여 표시한다.
- **PixelRelative (mode 1):** $|A - B| / \max(A, \epsilon) \times \text{amplify}$ — A를 기준으로 한 상대 차이를 계산한다.
- **FalseColor (mode 2):** 차이값을 5단계 컬러 램프에 매핑한다.

FalseColor의 `falsecolor()` 함수는 0.0~1.0 범위의 입력값을 다음과 같이 변환한다:

구간	색상	의미
0.0~0.2	Blue → Cyan	미세한 차이
0.2~0.4	Cyan → Green	약한 차이
0.4~0.6	Green → Yellow	중간 차이
0.6~0.8	Yellow → Red	강한 차이
0.8~1.0	Red	매우 큰 차이

표 8: FalseColor 5단계 컬러 램프

4.2.4. SSIM Heatmap Shader

CPU에서 사전 계산된 SSIM dissimilarity 맵을 시각화하는 셰이더이다.

```
#version 150
uniform sampler2D u_tex;      // R32F 단채널 텍스처
uniform float u_amplify;
```

입력은 GL_R32F 포맷의 단채널 float 텍스처로, 각 픽셀의 값은 $(1 - SSIM) / 2$ 범위(0=동일, 0.5=완전히 다름)이다. 이 값을 falsecolor() 함수에 넣어 FalseColor 히트맵으로 표시한다.

4.3. 뷰포트 변환 수학

셰이더의 핵심은 화면 좌표(viewport UV)를 이미지 좌표(image pixel)로 변환하는 것이다.

4.3.1. 순방향 변환 (Shader)

```
screen_px = uv * view_size
img_px    = (screen_px - view_size / 2) / zoom - pan + image_size / 2
uv_img    = img_px / image_size
```

4.3.2. 역방향 변환 (CPU)

UI 오버레이(좌표 표시, 미니맵 등)에서는 이미지 픽셀 좌표를 화면 좌표로 변환해야 한다.

```
screen_px = (img_px + pan - image_size / 2) * zoom + view_size / 2
```

4.3.3. 좌표계 정의

좌표계	원점	범위
Screen space	뷰포트 좌상단 (0, 0)	$(0, 0) \rightarrow (\text{view_w}, \text{view_h})$
Image space	이미지 좌상단 (0, 0)	$(0, 0) \rightarrow (\text{img_w}, \text{img_h})$
NDC	화면 중앙 (0, 0)	$(-1, -1) \rightarrow (1, 1)$
Texture UV	좌하단 (0, 0)	$(0, 0) \rightarrow (1, 1)$

표 9: 좌표계 정의

4.4. FBO (Framebuffer Object)

각 ImagePanel 은 전용 FBO를 가지며, 셰이더 렌더링 결과를 이 FBO에 기록한 뒤 ImGui::Image() 로 화면에 표시한다.

4.4.1. FBO 구조체

```
struct FBO {
    GLuint fbo_id = 0;
    GLuint tex_id = 0;    // RGBA8 color attachment
    int    width  = 0;
    int    height = 0;

    bool ensure(int w, int h); // 크기 변경 시 재생성
    void bind()  const;        // 렌더 타겟으로 바인드
    void unbind() const;       // 기본 프레임버퍼로 복원
};
```

`ensure(w, h)` 메서드는 요청된 크기가 현재 FBO 크기와 다를 때만 텍스처와 프레임버퍼를 재생성한다. 매 프레임 호출해도 크기가 같으면 비용이 발생하지 않는다.

4.4.2. 렌더 흐름

1. `fbo.ensure(panel_w, panel_h)` — 패널 크기에 맞춰 FBO 준비
2. `fbo.bind()` — 렌더 타겟을 FBO로 전환
3. `glViewport(0, 0, panel_w, panel_h)` — 뷰포트 설정
4. 셰이더 uniform 설정 + `quad.draw()` — full-{}screen quad 렌더링
5. `fbo.unbind()` — 기본 프레임버퍼로 복원
6. `ImGui::Image(fbo.tex_id, size)` — FBO 텍스처를 ImGui 위젯으로 표시

4.5. ScreenQuad

`ScreenQuad` 는 화면 전체를 커버하는 사각형 메쉬로, 모든 이미지 렌더링의 기하 구조를 제공한다.

```
struct ScreenQuad {
    GLuint vao = 0, vbo = 0;
    void init(); // VAO/VBO 생성 및 정점 업로드
    void draw() const; // glDrawArrays(GL_TRIANGLES, 0, 6)
};
```

정점 레이아웃은 다음과 같다. 각 정점은 위치(2 float)와 UV(2 float)로 구성된다.

#	Position (NDC)	UV
0	(-1, -1)	(0, 0)
1	(1, -1)	(1, 0)
2	(1, 1)	(1, 1)
3	(-1, -1)	(0, 0)
4	(1, 1)	(1, 1)
5	(-1, 1)	(0, 1)

표 10: ScreenQuad 정점 데이터 (삼각형 2개, 6 vertices)

4.6. 텍스처 관리

`gl_texture.h`에 정의된 텍스처 유틸리티 함수들은 CPU측 픽셀 데이터를 GPU 텍스처로 업로드한다.

4.6.1. 업로드 함수

함수	GL 포맷	용도
<code>gl_upload_texture()</code>	<code>GL_RGBA8</code>	8비트 LDR 이미지 (PNG, JPEG 등)
<code>gl_upload_texture_f32()</code>	<code>GL_RGBA32F</code>	32비트 HDR 이미지 (.hdr)
<code>gl_upload_texture_r32f()</code>	<code>GL_R32F</code>	SSIM 히트맵 (단채널 float)
<code>gl_delete_texture()</code>	—	텍스처 해제 및 핸들 초기화

표 11: 텍스처 업로드 함수

4.6.2. 필터링 전략

텍스처 필터링은 줌 레벨에 따라 동적으로 전환된다.

- 줌 $\geq 1x$: `GL_NEAREST` — 픽셀 경계가 선명하게 보이도록 최근접 보간
- 줌 $< 1x$: `GL_LINEAR_MIPMAP_LINEAR` — 축소 시 앨리어싱 방지를 위한 선형 밍맵 보간

모든 텍스처는 `GL_CLAMP_TO_EDGE` wrap 모드를 사용하여 경계에서의 아티팩트를 방지한다.

4.7. 전체 렌더링 흐름

그림 3은 단일 프레임에서 `ImageEntry`로부터 최종 화면 표시까지의 렌더링 파이프라인을 보여준다.

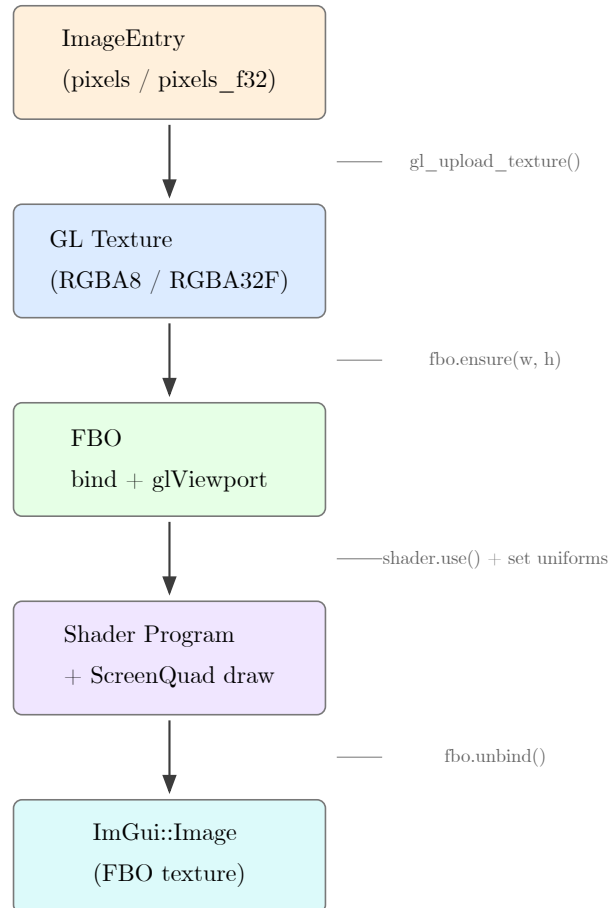


그림 3: 단일 프레임 렌더링 파이프라인

4.7.1. 단일 이미지 렌더링 절차

1. **Fit 계산:** `vp.fit == true` 이면 `zoom = min(view_w / img_w, view_h / img_h)`
2. **FBO 준비:** `fbo.ensure(panel_w, panel_h)`
3. **FBO 바인드:** `fbo.bind()` → `glViewport(0, 0, panel_w, panel_h)`
4. **텍스처 바인드:** `glActiveTexture(GL_TEXTURE0)`, `glBindTexture(GL_TEXTURE_2D, texture_id)`
5. **필터 설정:** 줌에 따라 `GL_NEAREST` 또는 `GL_LINEAR_MIPMAP_LINEAR` 전환
6. **Uniform 설정:** `u_image_size`, `u_view_size`, `u_zoom`, `u_pan`, `u_channel`
7. **드로우:** `quad.draw()` — 6 vertices, 2 triangles
8. **FBO 언바인드:** `fbo.unbind()`
9. **팬 클램프:** 이미지가 최소 50px 이상 보이도록 팬 값 제한
10. **화면 표시:** `ImGui::Image(fbo.tex_id, avail_size)`

4.7.2. Diff 렌더링 절차

Diff 렌더링은 단일 이미지와 유사하나, 두 텍스처를 동시에 바인드한다.

1. 이미지 A를 `GL_TEXTURE0`, 이미지 B를 `GL_TEXTURE1` 에 바인드

2. `DIFF_FRAG_SRC` 셰이더 사용
3. `u_diff_mode` (0/1/2)와 `u_amplify` uniform 전달
4. 나머지는 단일 이미지 렌더링과 동일

5. 이미지 처리

본 장에서는 av가 이미지를 로딩하고, GPU 텍스처로 업로드하며, LRU 캐시를 통해 효율적으로 관리하는 과정을 설명한다.

5.1. stb_image 로딩

av는 단일 헤더 라이브러리인 stb_image를 사용하여 다양한 래스터 포맷을 통합 로딩한다. 핵심 인터페이스는 다음과 같다.

```
// image_loader.h
bool load_image(const std::string& path, ImageEntry& entry);
void free_image(ImageEntry& entry);
```

load_image()는 파일 경로를 받아 ImageEntry 구조체에 디코딩 결과를 채운다. 내부적으로 SDR/HDR 경로가 분기된다.

5.1.1. ImageEntry 구조체

```
struct ImageEntry {
    std::string      path;
    std::vector<uint8_t> pixels;      // CPU RGBA8
    std::vector<float> pixels_f32;   // CPU RGBA32F (HDR)
    std::vector<uint16_t> pixels_orig; // PPM 원본 (P2/P3/P5/P6)
    int width       = 0;
    int height      = 0;
    int channels     = 0;
    int ppm_maxval  = 0;              // PPM maxval
    unsigned int texture_id = 0;      // GLuint
    bool loaded     = false;
    bool is_hdr     = false;
};
```

pixels와 pixels_f32는 각각 SDR과 HDR 이미지의 CPU 측 원본 데이터를 보유한다. GPU 업로드 후에도 SSIM 계산 등을 위해 CPU 데이터를 유지한다.

5.1.2. 지원 포맷

포맷	설명	비고
PNG	8/16-bit, 알파 지원	최우선 지원 포맷
JPEG	표준 손실 압축	EXIF 방향 자동 적용
BMP	비트맵	Windows 호환
TGA	Targa	알파 채널 지원
HDR	Radiance RGBE	float 텍스처로 업로드
PIC	Softimage PIC	—
PNM (P5/P6)	PGM / PPM Binary	av 자체 파서 (<code>try_load_ppm_binary</code>), 16-bit 원본값 보존
PNM (P2/P3)	PGM / PPM ASCII	av 자체 파서 (<code>try_load_ppm_ascii</code>), 원본값 보존

표 12: 지원 포맷 목록

5.1.3. 로딩 흐름

이미지 로딩은 PPM 자체 파서(ASCII → Binary 순)를 우선 시도하고, 실패 시 `stb_image`로 fallback 한다.

- PPM ASCII 경로:** `try_load_ppm_ascii()` 우선 시도 → P2(Grayscale) / P3(RGB) ASCII 파싱
 - 매직 넘버(P2 / P3) 확인 후 # 주석 행 스킵
 - width, height, maxval 파싱 후 ASCII 정수값을 순차 읽기
 - 원본 값을 `pixels_orig` (`uint16_t`)에 보존하고, 8-bit RGBA로 정규화하여 `pixels`에 저장
 - `ppm_maxval`에 원본 최대값 기록 (예: 1023, 65535 등)
- PPM Binary 경로:** `try_load_ppm_binary()` → P5(Grayscale) / P6(RGB) Binary 파싱
 - 매직 넘버(P5 / P6) 확인 후 헤더 파싱 (기존 `ppm_skip_ws_comments()` 재사용)
 - maxval 뒤 정확히 1바이트 whitespace 스킵 후 `f.read()`로 전체 픽셀 데이터 일괄 읽기
 - maxval ≤ 255: 채널당 1바이트, maxval > 255: 채널당 2바이트 (빅엔디안)
 - P5 grayscale은 R=G=B로 복제하여 `pixels_orig` (RGB 3ch)에 저장
 - maxval > 255일 때 `pixels_f32` (RGBA32F) 생성, GL_RGBA32F 텍스처로 업로드
- SDR 경로:** `stbi_load()` → RGBA8 (`uint8_t` 배열, 4채널 강제)
- HDR 경로:** `stbi_loadf()` → RGBA32F (`float` 배열, 4채널)
- `stbi_is_hdr()` 호출로 HDR 여부를 사전 판별하여 SDR/HDR 경로를 분기한다.

5.2. HDR 지원

HDR 이미지는 일반 8-bit 범위를 초과하는 넓은 다이내믹 레인지를 제공한다. av의 HDR 처리 과정은 다음과 같다.

1. `stbi_is_hdr(path)`: 파일 헤더를 읽어 HDR 여부 감지
2. `stbi_loadf()`: `.hdr` 파일을 RGBA 4채널 `float` 배열로 디코딩
3. `pixels_f32` 벡터에 CPU 측 데이터 저장
4. `upload_rgba_f32()`: `GL_RGBA32F` 내부 포맷으로 GPU 텍스처 업로드

GPU 업로드 시 HDR 텍스처는 mip맵을 생성하지 않으며 (`GL_LINEAR` 필터), 톤 매핑은 프래그먼트 셰이더 단에서 처리할 수 있다. 현재 구현에서는 linear 값 그대로 표시한다.

5.2.1. GPU 업로드 헬퍼

```
static GLuint upload_rgba8(const uint8_t* pixels, int w, int h);
static GLuint upload_rgba_f32(const float* pixels, int w, int h);
```

항목	RGBA8	RGBA32F	비고
내부 포맷	<code>GL_RGBA8</code>	<code>GL_RGBA32F</code>	—
필터	Mipmap Linear	Linear	HDR는 mip맵 없음
래핑	Clamp to Edge	Clamp to Edge	동일
Border Color	0.15 gray	0.15 gray	이미지 외곽 배경

표 13: SDR / HDR 텍스처 업로드 파라미터 비교

5.3. LRU 캐시

av는 최대 8개의 이미지를 GPU 텍스처와 함께 메모리에 유지하는 LRU(Least Recently Used) 캐시를 제공한다.

```
class ImageCache {
public:
    static constexpr int MAX_ENTRIES = 8;
    ImageEntry* get(const std::string& path);
    void clear();
    int size() const;
private:
    struct CacheEntry {
        ImageEntry image;
```

```

    uint64_t    last_access = 0;
};
std::vector<CacheEntry> entries_;
uint64_t        access_counter_ = 0;
void evict_lru();
};
extern ImageCache g_image_cache;

```

5.3.1. LRU 동작 시나리오

`get(path)` 호출 시 다음 순서로 동작한다.

1. 캐시 내 `path` 일치 항목 검색
2. **캐시 히트**: `last_access` 를 현재 `access_counter_++` 로 갱신 후 포인터 반환
3. **캐시 미스**: `load_image()` 호출로 디스크에서 로딩
4. 캐시가 가득 찼으면 (`entries_.size() >= MAX_ENTRIES`) `evict_lru()` 호출
5. `evict_lru()`: `last_access` 가 가장 작은 엔트리를 `free_image()` 로 해제 후 제거
6. 새 엔트리를 캐시에 추가하고 포인터 반환

5.3.2. 캐시 도식

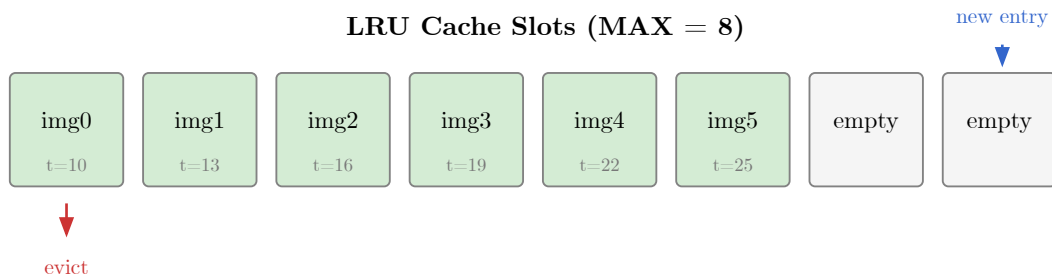


그림 4: LRU 캐시 슬롯 구조 (8 엔트리)

5.4. 메모리 관리

이미지 데이터는 CPU와 GPU 양쪽에 존재할 수 있다.

- **CPU 픽셀 데이터**: `pixels` (SDR) 또는 `pixels_f32` (HDR) 벡터
- **GPU 텍스처**: `texture_id` 가 가리키는 OpenGL 텍스처 오브젝트

SSIM 계산 시 CPU 측 픽셀 데이터가 필요하므로, 비교 기능을 사용할 경우 CPU 데이터를 해제하면 안 된다. `free_image()` 호출 시 `glDeleteTextures()` 와 함께 벡터를 비운다.

```
void free_image(ImageEntry& entry) {
    if (entry.texture_id) {
        glDeleteTextures(1, &entry.texture_id);
        entry.texture_id = 0;
    }
    entry.pixels.clear();
    entry.pixels_f32.clear();
    entry.loaded = false;
}
```

5.5. 이미지 회전

av는 CPU 기반의 90° 회전 기능을 제공한다. **r** 키는 시계방향(CW), **Ctrl+R** 키는 반시계방향(CCW)으로 모든 로드된 이미지를 동시에 90° 회전한다.

5.5.1. 인터페이스

```
// image_loader.h
void rotate_image_cw (ImageEntry& entry);    // 시계방향 90°
void rotate_image_ccw(ImageEntry& entry);    // 반시계방향 90°
```

두 함수 모두 `ImageEntry` 를 제자리에서 수정한다. CPU 측 픽셀 데이터(`pixels` 또는 `pixels_f32`)와 GPU 텍스처를 모두 갱신한다.

5.5.2. 변환 공식

회전 전 이미지 크기: $w_{\text{old}} \times h_{\text{old}} \rightarrow$ 회전 후: $w_{\text{new}} = h_{\text{old}}, h_{\text{new}} = w_{\text{old}}$

방향	변환	수식
CW (시계방향)	$\text{new}[x][\text{old_h}-1-y] = \text{old}[y][x]$	$\text{dst_row} = x, \text{dst_col} = \text{old_h}-1-y$
CCW (반시계)	$\text{new}[\text{old_w}-1-x][y] = \text{old}[y][x]$	$\text{dst_row} = \text{old_w}-1-x, \text{dst_col} = y$

표 14: 90° 회전 변환 공식

5.5.3. 구현 세부

1. **데이터 타입 분기**: `LDR(uint8_t × 4, pixels)` / `HDR(float × 4, pixels_f32)` 모두 처리
2. **GPU 텍스처 재생성**: 기존 텍스처를 `glDeleteTextures()` 로 해제 후 `upload_rgba8()` 또는 `upload_rgba_f32()` 로 재업로드

3. 뷰포트 초기화: 회전 후 모든 뷰포트의 `fit = true`, `pan_x = pan_y = 0.0f` 로 초기화 (fit-to-window)

```
void rotate_image_cw(ImageEntry& entry) {
    if (!entry.loaded) return;
    const int old_w = entry.width, old_h = entry.height;
    const int new_w = old_h,      new_h = old_w;

    // LDR
    if (!entry.pixels.empty()) {
        std::vector<uint8_t> dst(new_w * new_h * 4);
        for (int y = 0; y < old_h; ++y)
            for (int x = 0; x < old_w; ++x) {
                int dst_row = x, dst_col = old_h - 1 - y;
                // 4채널 복사
                const uint8_t* sp = entry.pixels.data() + (y*old_w+x)*4;
                uint8_t* dp = dst.data() + (dst_row*new_w+dst_col)*4;
                dp[0]=sp[0]; dp[1]=sp[1]; dp[2]=sp[2]; dp[3]=sp[3];
            }
        entry.pixels = std::move(dst);
    }
    // HDR (pixels_f32 동일 구조)
    entry.width = new_w; entry.height = new_h;
    // GPU 재업로드
    glDeleteTextures(1, &entry.texture_id); entry.texture_id = 0;
    entry.texture_id = entry.pixels.empty()
        ? upload_rgba_f32(entry.pixels_f32.data(), new_w, new_h)
        : upload_rgba8   (entry.pixels.data(),      new_w, new_h);
}
```

6. 뷰포트 시스템

뷰포트 시스템은 이미지의 확대/축소(줌)와 이동(패닝)을 관리하며, 두 패널 간의 뷰포트 동기화를 지원한다.

6.1. 줌 레벨 시스템

av는 12단계의 이산적 줌 레벨을 사용한다. 모든 레벨은 2의 거듭제곱 배수이다.

```
inline constexpr float ZOOM_LEVELS[] = {
    0.125f, 0.25f, 0.5f,
    1.0f, 2.0f, 4.0f, 8.0f, 16.0f, 32.0f, 64.0f, 128.0f, 256.0f
};
inline constexpr int NUM_ZOOM_LEVELS = 12;
```

인덱스	배율	표시
0	0.125×	12.5%
1	0.25×	25%
2	0.5×	50%
3	1.0×	100%
4	2.0×	200%
5	4.0×	400%
6	8.0×	800%
7	16.0×	1600%
8	32.0×	3200%
9	64.0×	6400%
10	128.0×	12800%
11	256.0×	25600%

표 15: 12단계 줌 레벨 테이블

6.1.1. 줌 탐색

`nearest_zoom_index()` 함수가 현재 줌 값에서 가장 가까운 이산 레벨의 인덱스를 반환한다. `viewport_zoom_in()` 은 다음 상위 레벨로, `viewport_zoom_out()` 은 다음 하위 레벨로 이동한다. 줌 값은 항상 $[0.125, 256]$ 범위 내로 클램핑된다.

```
void viewport_zoom_in (ViewportState& v);    // 상위 레벨로
void viewport_zoom_out(ViewportState& v);    // 하위 레벨로
void viewport_set_zoom(ViewportState& v, float zoom); // 직접 설정
```

6.1.2. ViewportState 구조체

```
struct ViewportState {
    float zoom = 1.0f;    // 표시 배율
    float pan_x = 0.0f;   // 이미지 픽셀 단위 X 오프셋
    float pan_y = 0.0f;   // 이미지 픽셀 단위 Y 오프셋
    bool fit = true;      // fit-to-window 모드 여부
};
```

6.2. 패닝

`viewport_pan(v, dx, dy)` 는 이미지 픽셀 단위로 뷰포트를 이동한다.

```
void viewport_pan(ViewportState& v, float dx, float dy);
```

입력	동작	이동량
방향키	기본 이동	32 픽셀 (<code>pan_step</code>)
Shift + 방향키	단위 이동	<code>pan_step</code> 단위
마우스 드래그	좌클릭 드래그	delta 픽셀 직접 적용

표 16: 패닝 입력 방식

패닝이 발생하면 `fit` 플래그가 `false` 로 전환되어, 이후 뷰포트가 자동 맞춤 모드에서 벗어난다.

6.3. Fit-to-Window

`viewport_fit()` 함수는 이미지를 뷰포트에 맞추어 표시한다.

```
void viewport_fit(ViewportState& v,
    int img_w, int img_h,
    int view_w, int view_h);
```

계산 과정:

$$\text{zoom} = \min\left(\frac{w_{\text{view}}}{w_{\text{img}}}, \frac{h_{\text{view}}}{h_{\text{img}}}\right)$$

산출된 줌 값은 `ZOOM_LEVELS` 배열에서 해당 값 이하인 가장 가까운 레벨로 스냅된다. 패닝은 (0,0)으로 초기화되어 이미지가 뷰포트 중앙에 위치하며, `fit = true`가 설정된다.

6.4. 클램핑

`viewport_clamp_pan()`은 이미지가 뷰포트 밖으로 완전히 벗어나는 것을 방지한다.

```
void viewport_clamp_pan(ViewportState& v,
    int img_w, int img_h,
    int view_w, int view_h);
```

최소 50 픽셀의 오버랩 마진을 유지하여, 이미지의 일부가 항상 화면에 보이도록 보장한다. 클램핑 공식은 다음과 같다.

$$\max_x = \frac{w_{\text{img}}}{2} + \frac{w_{\text{view}}}{2 \cdot \text{zoom}} - \frac{\text{MARGIN}}{\text{zoom}}$$

6.5. 뷰포트 동기화

두 패널 모드에서 `S` 키를 눌러 뷰포트 동기화를 토글할 수 있다 (`sync_viewports` 플래그).

- **동기화 활성화:** `views[0]` (좌측 패널)이 마스터 역할을 하며, 줌/패닝 변경 시 `views[1]`에 즉시 복사된다.
- **동기화 비활성:** 각 패널이 독립적으로 줌/패닝을 유지한다.

이를 통해 두 이미지의 동일 영역을 정확히 비교할 수 있다.



그림 5: 뷰포트 동기화 모드 비교

7. 비교 엔진

비교 엔진은 두 이미지 간의 차이를 다양한 방식으로 시각화한다. GPU 셰이더 기반의 실시간 픽셀 차이 연산과 CPU 기반의 비동기 SSIM 계산을 모두 지원한다.

7.1. GPU Diff 렌더러

`DiffRenderer` 클래스는 두 텍스처를 입력받아 차이 결과를 화면에 렌더링한다.

```
class DiffRenderer {
public:
    bool init();
    void render(GLuint texA, GLuint texB,
               const ViewportState& view,
               int img_w, int img_h,
               int view_w, int view_h,
               DiffState::Mode mode, float amplify,
               ChannelMode channel);
    bool valid() const { return shader_.valid(); }
private:
    ShaderProgram shader_;
    ScreenQuad quad_;
};
```

`render()` 호출 시 내부 동작:

1. FBO(Framebuffer Object) 바인드
2. `shader_.use()` → uniform 값 설정 (`u_diff_mode`, `u_amplify`, `u_channel` 등)
3. `quad_.draw()`: 6개 정점의 풀스크린 쿼드 렌더링
4. FBO 언바인드

7.2. Diff 모드 3종 상세

프래그먼트 셰이더의 `u_diff_mode` uniform에 따라 세 가지 GPU 차이 모드가 적용된다.

7.2.1. PixelAbsolute (`u_diff_mode = 0`)

두 이미지 픽셀의 절대 차이를 계산한다.

```
vec4 diff = abs(a - b);
result = diff.rgb * u_amplify;
```

증폭 계수(`u_amplify`)를 곱하여 미세한 차이도 시각적으로 구분 가능하게 한다.

7.2.2. PixelRelative (`u_diff_mode = 1`)

원본 밝기 대비 상대적 차이를 계산한다.

```
vec4 diff = abs(a - b) / max(a + vec4(0.001), vec4(0.001));
result = diff.rgb * u_amplify;
```

분모에 0.001을 더해 영점 나눗셈을 방지한다. 밝은 영역과 어두운 영역의 차이를 균일하게 비교할 수 있다.

7.2.3. FalseColor (`u_diff_mode = 2`)

차이 강도를 열지도(heatmap) 색상으로 매핑한다.

```
vec3 falsecolor(float t) {
    if (t < 0.25) c = mix(navy, blue, t * 4.0);
    else if (t < 0.5) c = mix(blue, green, (t-0.25) * 4.0);
    else if (t < 0.75) c = mix(green, yellow, (t-0.5) * 4.0);
    else c = mix(yellow, red, (t-0.75) * 4.0);
}
```

범위	색상 전이	의미
0.00 ~ 0.25	남색 → 파랑	최소 차이
0.25 ~ 0.50	파랑 → 초록	낮은 차이
0.50 ~ 0.75	초록 → 노랑	중간 차이
0.75 ~ 1.00	노랑 → 빨강	최대 차이

표 17: FalseColor 열지도 색상 매핑

7.2.4. Diff 모드 비교 요약

모드	설명	적합한 용도
PixelAbsolute	$ A - B \times \text{amplify}$	전반적 차이 확인
PixelRelative	$ A - B _A \times \text{amplify}$	상대적 차이 (밝기 무관)
FalseColor	강도 기반 열지도 매핑	차이 분포 시각화
SSIM	CPU 비동기 계산 (지각적 유사도)	구조적 유사도 평가

표 18: 비교 모드 요약

7.3. CPU SSIM 비동기 계산

SSIM(Structural Similarity Index)은 인간의 시각 인지와 더 잘 부합하는 이미지 품질 지표이다. av는 CPU에서 비동기적으로 SSIM을 계산한다.

7.3.1. SSIMComputer 클래스

```
class SSIMComputer {
public:
    void compute(const ImageEntry& a,
                 const ImageEntry& b,
                 std::function<void(SSIMResult)> callback);

    void cancel();
    bool is_running() const { return running_.load(); }
private:
    std::jthread      thread_;
    std::atomic<bool> running_{false};
    std::atomic<bool> cancel_requested_{false};
};
```

7.3.2. SSIMResult 구조체

```
struct SSIMResult {
    float      score      = 0.0f; // 0(다름) ~ 1(동일)
    std::vector<float> heatmap;    // 픽셀별 비유사도 [0,1]
    int        w          = 0;
    int        h          = 0;
    bool       success    = false;
};
```

7.3.3. SSIM 공식

SSIM은 두 윈도우 x, y 에 대해 다음과 같이 정의된다.

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

여기서:

- μ_x, μ_y : 각 윈도우의 평균 휘도
- σ_x^2, σ_y^2 : 분산
- σ_{xy} : 공분산
- $C_1 = (0.01 \times 255)^2 \approx 6.5025$
- $C_2 = (0.03 \times 255)^2 \approx 58.5225$

7.3.4. SSIM 상수

```
constexpr int    SSIM_WIN = 11;        // 11×11 윈도우
constexpr double SSIM_C1  = 6.5025;    // (0.01 * 255)2
constexpr double SSIM_C2  = 58.5225;  // (0.03 * 255)2
```

윈도우는 $\sigma = 1.5$ 인 11×11 가우시안 커널을 사용한다.

7.3.5. 비동기 계산 과정

1. `compute()` 호출 시 `std::jthread` 생성 (백그라운드 실행)
2. 이미지를 휘도로 변환: $L = 0.2126R + 0.7152G + 0.0722B$
3. 11×11 가우시안 윈도우를 슬라이딩하며 SSIM 계산
4. 비유사도(dissimilarity) 변환: $d = \frac{1 - \text{SSIM}}{2} \rightarrow [0, 1]$ 범위
5. 결과를 `heatmap` 벡터에 저장
6. `cancel_requested_` 플래그를 주기적으로 확인하여 조기 취소 지원
7. 완료 시 콜백 호출 → 메인 스레드에서 `gl_upload_texture_r32f()` 로 단채널 float 텍스처 업로드

7.4. 채널 분리

`ChannelMode` enum으로 개별 채널을 분리 표시할 수 있다.

```
enum class ChannelMode { RGB = 0, Red = 1, Green = 2, Blue = 3 };
```

모드	u_channel	표시
RGB	0	전체 컬러 (기본)
Red	1	R 채널만 그레이스케일로 표시
Green	2	G 채널만 그레이스케일로 표시
Blue	3	B 채널만 그레이스케일로 표시

표 19: 채널 분리 모드

프래그먼트 셰이더에서 `u_channel` uniform 값에 따라 해당 채널 값만 추출하여 그레이스케일로 렌더링한다.

7.5. 비교 모드 시각화

그림 6 은 av가 제공하는 네 가지 비교 모드의 개념적 시각화이다.



그림 6: 비교 엔진 모드 시각화

7.5.1. 픽셀 그리드 오버레이

줌 배율이 $16\times$ 이상일 때, 프래그먼트 셰이더는 개별 픽셀 경계에 그리드 라인을 오버레이한다.

```
if (u_zoom >= 16.0) {
    vec2 frac_px = fract(img_px);
    vec2 grid_dist = min(frac_px, 1.0 - frac_px);
    float line_w = 1.0 / u_zoom;
    float grid = smoothstep(line_w, 0.0,
                            min(grid_dist.x, grid_dist.y));
    out_color.rgb = mix(out_color.rgb, vec3(0.5), grid * 0.4);
}
```

이 오버레이는 고배율에서 개별 픽셀을 정확히 식별할 수 있도록 도와주며, diff 모드와 일반 이미지 표시 모드 모두에서 동작한다.

8. 사용자 인터페이스

Advanced Pixel Lens의 사용자 인터페이스는 SDL3 윈도우 위에 ImGui를 레이어링하여 구성된다. 메뉴바, 상태바, 팝업 등 모든 UI 요소는 ImGui로 렌더링되며, 이미지 자체는 OpenGL 텍스처로 직접 그려진다.

8.1. 패널 레이아웃

패널 레이아웃은 로드된 이미지 수와 diff 모드 활성화 여부에 따라 자동으로 결정된다.

모드	조건	배치
1-Panel	Image A만 로드	전체 너비에 Image A 표시
2-Panel	A+B 로드, diff=None	좌측 50% A, 우측 50% B (side-by-side)
3-Panel	A+B 로드, diff 활성화	33%씩 A B Diff 세 패널

표 20: 패널 레이아웃 모드별 조건과 배치

아래 다이어그램은 세 가지 패널 구성을 도식화한 것이다.

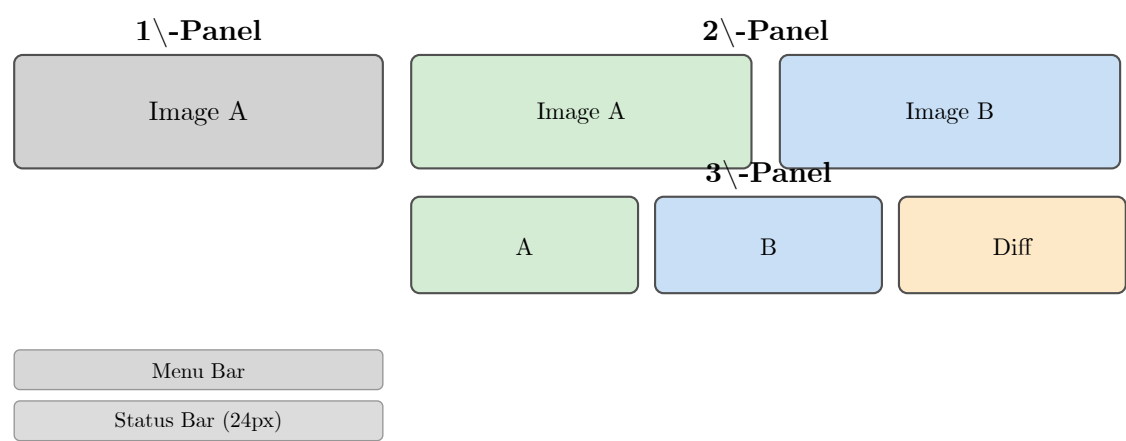


그림 7: 패널 레이아웃 다이어그램

예약 공간:

- **메뉴바:** 동적 높이 (ImGui 자동 계산)
- **상태바:** 24px 고정 높이 (UI 표시 상태일 때)

8.2. 메뉴바

ImGui 메인 메뉴바는 네 개의 메뉴로 구성된다.

File 메뉴:

- Open Image A... (**O**) – 파일 열기 다이얼로그 (Image A)
- Open Image B... (**Shift+O**) – 파일 열기 다이얼로그 (Image B)
- Open Image (**Shift+Cmd+O**) – 활성 패널용 네이티브 파일 다이얼로그 직접 호출
- Separator
- Save (**Shift+Cmd+S**) – 컨텍스트 인식 Save 다이얼로그 토글
- Separator
- Quit (**Q**) – 애플리케이션 종료

View 메뉴:

- Fit to Window (**F**) – 윈도우에 맞춤
- 1:1 Pixel (**1**) – 100% 원본 크기
- Separator
- Sync Viewports (**S**) – 체크박스 토글
- Separator
- Pathfinder Image (**P**) – 이미지 미니맵 토글
- Pathfinder Schematic (**Ctrl+P**) – 배선도 미니맵 토글
- Show Image Info (**I**) – 정보 팝업 체크박스 토글
- Show Pixel Info (**V**) – 커서 픽셀 값 풍선말 토글
- Show Histogram (**Ctrl+H**) – Histogram 창 토글 (상호 배타)
- Show H-Line Cut (**Ctrl+L**) – H-Line Cut 창 토글 (상호 배타)
- Show V-Line Cut (**Ctrl+Y**) – V-Line Cut 창 토글 (상호 배타)
- Show Statistics (**Ctrl+S**) – Statistics 창 토글 (상호 배타)

Diff 메뉴:

- Off (**Ctrl+D**)
- Absolute (**Ctrl+3**)
- Relative (**Ctrl+4**)
- FalseColor (**Ctrl+5**)
- SSIM (**Ctrl+6**)
- Separator
- Amplify 슬라이더 (0.5~50.0×)

우측 정렬:

- 실시간 FPS 표시 (disabled 텍스트 색상)

8.3. 상태바

하단 상태바는 24px 높이로 고정되며, 배경색은 `RGB(0.12, 0.12, 0.12)` 이다.

항목	표시 형식
줌 배율	Zoom: NN% (활성 패널 기준)
Image A 정보	A: WIDTHxHEIGHT 또는 A: - (미로드 시)
Image B 정보	B: WIDTHxHEIGHT 또는 B: - (미로드 시)
Diff 모드	모드명(Abs/Rel/FalseColor/SSIM) + xN.N 배율
SSIM 점수	점수별 색상: ≥ 0.99 초록, ≥ 0.90 노랑, < 0.90 빨강
동기화 상태	Sync: on 또는 Sync: off
FPS	우측 정렬, disabled 텍스트 색상

표 21: 상태바 정보 항목

각 항목은 `|` 구분자로 분리되며, SSIM 점수는 연산 중에는 `SSIM: computing...` (연한 파란색)으로 표시된다.

8.4. 이미지 정보 팝업

`I` 키로 토글되는 플로팅 팝업으로, 로드된 이미지의 메타데이터와 각 패널의 현재 줌 상태를 표시한다.

- 윈도우 크기: 너비 600px, 높이 자동 (`SetNextWindowSize({600, 0})`)
- 폰트 배율: `SetWindowFontScale(2.0f)` — 기본 폰트의 2배 크기로 렌더링
- 이미지 정보 항목:
 - 파일 경로 (전체)
 - 해상도: `WIDTHxHEIGHT`
 - 채널 수
 - HDR 여부: `true` / `false`
- Separator 이후 각 패널의 줌 상태:
 - Fit 모드: A Zoom: `Fit-to-Window`
 - 수동 줌: A Zoom: `200.0%` Pan: `(12, -5)`

8.5. 줌 HUD

줌 배율이 변경될 때 화면 상단 중앙에 캡슐 형태의 오버레이가 표시된다.

- 표시 시간: 줌 변경 후 5초간 (`zoom_hud_timer = 5.0f`)

- **페이드 아웃:** 마지막 1초 동안 선형 감쇠
- **캡슐 스타일:** 검정 배경 (`alpha = 180 × fade`), 흰색 텍스트
- **패딩:** 수평 16px, 수직 8px
- **위치:** 윈도우 상단 중앙에서 12px 아래
- **형식:** $\geq 1 \times$ 시 `%.0fx` (예: `2x`), $< 1 \times$ 시 `%.2fx` (예: `0.50x`)

8.6. 경계 렌더링 (Border)

ImGui DrawList를 사용하여 각 패널의 이미지 경계를 시각적으로 표시한다.

패널	기본 색상	ABGR 값
Image A	마젠타 (Magenta)	<code>0xE6FF00FF</code>
Image B	노랑 (Yellow)	<code>0xE600FFFF</code>
Diff	시안 (Cyan)	<code>0xE6FFFF00</code>

표 22: 패널별 경계 색상 (ImGui ABGR 포맷, `alpha=0xE6=230`)

렌더링 구조:

- **두께:** 메인 라인 2px + 그림자 오프셋 3px (+1,+1)
- **그림자 색상:** `RGB(0,0,0, alpha=160)`
- **좌표 변환:** `screen_px = (img_px + pan - half_img) × zoom + half_view`
- **클리핑:** 수직/수평 에지 모두 뷰포트 범위로 클리핑
- **커스텀:** `-bc` CLI 옵션으로 6자리 hex 색상 지정 가능
- **토글:** `B` 키로 모든 패널 테두리를 런타임에 on/off 전환. `state.show_borders` 가 `false` 면 `draw_image_border()` 호출과 포그라운드 패널 보더 드로잉이 모두 스킵된다.
- **영속화:** 종료 시 `av.ini` 에 `show_borders=0|1` 로 저장, 시작 시 자동 로드. `-nb` CLI 옵션은 INI 값을 override.

8.7. Pathfinder (미니맵)

좌측 패널(`panel_idx=0`)의 좌측 하단에 미니맵을 표시하여 현재 뷰포트의 전체 이미지 내 위치를 파악할 수 있게 한다.

모드:

- Mode 1 (Image, `P` 키) — 이미지 썸네일 미니맵
- Mode 2 (Schematic, `Ctrl+P`) — 배선도 미니맵 (회색 배경 + 노란 반투명 채움)

표시 조건:

- `pathfinder_mode ≠ 0` (기본 Mode 1)
- Fit 모드가 아닐 때 (줌 인 상태)

크기 계산:

- 최대 너비: 150px 또는 패널 너비의 20% 중 작은 값
- 종횡비: 이미지 종횡비 유지
- 최대 높이: 패널 높이의 30%
- 여백: 에지에서 8px

스타일:

- 배경: 반투명 검정 (`alpha=180`), 라운드 4px
- 테두리: 흰색 (`alpha=100`), 1px 두께
- 뷰포트 표시기: 노란색 사각형 (`RGB(255,220,0)`, `alpha=220`), 1.5px 두께

현재 뷰포트 범위를 `[0,1]` 정규화 좌표로 변환한 뒤 미니맵에 매핑하여 가시 영역을 사각형으로 표시한다.

8.8. 픽셀 값 표시

줌 배율이 $32\times$ 이상일 때 자동으로 픽셀 그리드와 값이 표시된다.

픽셀 그리드:

- 수직/수평선: 각 픽셀 경계에 정렬
- 색상: 흰색 (`alpha=50`)
- 뷰포트 범위로 클리핑

RGB 모드 표시:

- 세 줄 스택: R(상), G(중), B(하)
- R 색상: `RGB(255,100,100, alpha=230)`
- G 색상: `RGB(100,255,100, alpha=230)`
- B 색상: `RGB(100,130,255, alpha=230)`

단일 채널 모드: 해당 채널 색상으로 단일 값 표시

형식 (우선순위 순):

- PPM 원본 (`pixels_orig` 존재 시): 정수 ($0\sim\text{maxval}$ 범위, 예: 0-1023)
- HDR (`float32`): `%.2f` (형식 토크 대상 아님)
- LDR (`uint8`): 정수 ($0\sim255$)

픽셀값 형식 토크 (`Ctrl+X`): 정수 픽셀값의 표시 형식을 세 가지로 순환 전환한다. HDR float 값은 항상 소수점 형식을 유지한다.

- Decimal (기본): 128
- Hex 0x: 0x80 (C-style)
- Hex h: 80h (Intel/ASM-style)

`fmt_int_pixel()` 헬퍼 함수가 `PixelFormat` enum에 따라 `snprintf` 형식을 선택한다. Diff 패널의 `uint8` 차이값에도 동일한 형식이 적용된다.

글꼴 크기: `zoom × 0.25f` (최대 20.0f), 픽셀 중심에 배치, 그림자 오프셋 (+1,+1)

8.9. 픽셀 값 풍선말 (Pixel Balloon)

V 키로 `show_pixel_info` 를 토글하면, 마우스 커서가 위치한 픽셀의 좌표와 RGB 값을 풍선말 (balloon) 형태로 표시한다.

동작 원리:

- 마우스 화면 좌표에서 현재 활성 패널(1/2/3-패널 레이아웃)을 판별
- screen → image 좌표 변환 수행 (뷰포트 역변환)
- 이미지 범위 내 좌표에서 픽셀 값 샘플링

풍선말 스타일:

- 배경: `IM_COL32(20, 20, 20, 200)`, 라운드 반지름 6px
- 테두리: `IM_COL32(80, 80, 80, 180)`
- 좌표 행: 흰색 (123, 456)
- RGB 행: R/G/B 각 채널 색상으로 개별 표시 (우선순위 순)
 - PPM 원본 (pixels_orig 존재 시): R:512 G:1023 B:0 (원본 maxval 기준)
 - HDR (float32): R:1.234 G:0.562 B:0.012 (형식 토글 대상 아님)
 - LDR (uint8): R:255 G:128 B:64
- Diff 패널에서도 PPM 원본값 기준으로 차이를 계산하여 표시
- `Ctrl+X` 형식 토글이 풍선말에도 동일하게 적용 (예: `R:0x80 G:0xFF B:0x40`).
`fmt_balloon_pixel()` 헬퍼가 prefix 포함 포매팅 담당

Edge Clamping: 풍선말이 화면 밖으로 나가지 않도록 위치를 자동 보정한다.

8.10. 드래그 줌 (Drag-to-Zoom)

마우스 우클릭 드래그로 영역을 선택하여 해당 영역으로 줌 인한다.

- **최소 선택 크기:** 5px × 5px
- **드래그 중 시각 피드백:**
 - 반투명 흰색 채움 (alpha=25%)

- ▶ 노란색 테두리 (`alpha=220` , `1.5px`)
- **드래그 해제 시:**
 - ▶ 선택 영역에 맞는 최대 2^n 줌 레벨 자동 계산
 - ▶ 선택 영역을 뷰포트 중심으로 이동

8.11. Statistics 창 (Ctrl+S)

`Ctrl+S` 키(또는 View 메뉴 → Show Statistics)로 토글하는 `render_stats_window()` 함수가 Image A, Image B, Diff $|A-B|$ 세 섹션의 채널별 통계를 ImGui 테이블로 표시한다.

테이블 구조: 5개 컬럼으로 구성된다.

컬럼	R	G	B	비고
Metric (width 2.0)	1.2	1.2	1.2	컬럼 weight 비율
Description (width 4.0)	—	—	—	<code>TextWrapped</code> 자동 줄바꿈

표 23: Statistics 테이블 컬럼 구성

기본 통계 (13개): Image A / B 공통으로 표시된다.

Metric	Description
Pixels	Total number of pixels in the image (width × height)
Min	Smallest pixel value in the channel
Max	Largest pixel value in the channel
Dynamic Range	Difference between max and min values (Max − Min)
Mean	Average pixel value: sum of all values divided by pixel count
Std Dev	Standard deviation: measures the spread of values around the mean
Variance	Square of standard deviation: average squared deviation from the mean
Median	Middle value when all pixels are sorted; more robust to outliers than mean
Skewness	Asymmetry of the distribution: 0=symmetric, positive=right tail, negative=left tail
Kurtosis (excess)	Tail heaviness vs. normal distribution: 0=normal, positive=heavy tails, negative=light tails
Energy	Mean of squared pixel values (second moment about zero)
RMS	Root Mean Square: square root of energy; represents overall signal magnitude
Entropy (bits)	Shannon entropy: information content in bits; higher values indicate more complex/random data

표 24: 기본 통계 13개 및 Description

Diff 전용 통계 (4개): Image A와 Image B가 모두 로드된 경우에만 Diff 섹션에 추가 표시된다.

Metric	Description
MSE	Mean Squared Error: average of squared differences per pixel; lower = more similar
PSNR	Peak Signal-to-Noise Ratio in dB; higher = better quality (typical range: 30–50 dB)
MAE	Mean Absolute Error: average of absolute differences per pixel; lower = more similar
Max Error	Largest single-pixel absolute difference between image A and image B

표 25: Diff 전용 통계 4개 및 Description

8.12. 차트/통계 창 상호 배타적 토글

Histogram, H-Line Cut, V-Line Cut, Statistics 4개 창은 상호 배타적으로 동작한다. 하나를 열면 나머지 세 창이 자동으로 닫힌다. 이 동작은 키보드 단축키(`app.cpp`)와 메뉴바(`main_window.cpp`) 모두에서 동일하게 적용된다.

창	단축키	토글 시 동작
Histogram	<code>Ctrl+H</code>	나머지 3개 창(<code>hline_cut</code> , <code>vline_cut</code> , <code>stats</code>) 닫힘
H-Line Cut	<code>Ctrl+L</code>	나머지 3개 창(<code>histogram</code> , <code>vline_cut</code> , <code>stats</code>) 닫힘
V-Line Cut	<code>Ctrl+Y</code>	나머지 3개 창(<code>histogram</code> , <code>hline_cut</code> , <code>stats</code>) 닫힘
Statistics	<code>Ctrl+S</code>	나머지 3개 창(<code>histogram</code> , <code>hline_cut</code> , <code>vline_cut</code>) 닫힘

표 26: 차트/통계 창 상호 배타적 토글

8.13. Histogram 창 (Ctrl+H)

`Ctrl+H` 키(또는 View 메뉴 → Show Histogram)로 토글하는 `render_histogram_window()` 함수가 Image A, Image B, Diff $|A - B|$ 세 섹션의 R/G/B 채널별 히스토그램을 표시한다.

창 크기: 메인 뷰포트의 $90\% \times 85\%$ (첫 표시 시)

히스토그램 계산:

- 256 빈(bin) $\times$ 3 채널 (R/G/B) 각각 독립 집계
- Diff 섹션: Image A와 B의 채널별 절대 차이 $|A[i] - B[i]|$ 를 256 빈으로 집계

Y축 스케일: 로그 스케일 (`std::log1p`) 적용. 최대 로그값(`max_log`)으로 정규화하여 막대 높이를 결정.

Y축 라벨: 25/50/75/100% 위치에 실제 카운트 값을 쉼표 포맷(예: `1,234,567`)으로 표시. 라벨 폭에 맞춰 좌측 여백(`y_margin`)을 동적으로 계산.

X축 틱: `{0, 32, 64, 96, 128, 160, 192, 224, 255}` 고정 9개.

채널	색상 (RGBA)	비고
R	(255, 60, 60, 180)	붉은 막대
G	(60, 255, 60, 180)	녹색 막대
B	(60, 60, 255, 180)	파란 막대

표 27: Histogram 채널 막대 색상

스타일:

- 차트 배경: (30, 30, 30, 220)
- 격자선: 25/50/75/100% 위치, (60, 60, 60, 120)
- 테두리: (80, 80, 80, 180)

호버 툴팁: 마우스가 차트 영역 위에 있을 때 수직 크로스헤어 3개(채널별, 흰색 alpha=120)와 함께 현재 bin 번호, R/G/B 카운트(심표 포맷)를 툴팁으로 표시한다.

8.14. H-Line Cut 창 (Ctrl+L)

Ctrl+L 키로 토글하는 `render_hline_cut_window()` 함수가 현재 뷰포트 중앙에 해당하는 수평 라인의 픽셀값을 R/G/B 채널별 라인 차트로 표시한다.

창 크기: 메인 뷰포트의 90% × 85% (첫 표시 시)

라인 선택: 뷰포트 `pan_y`를 기반으로 계산.

$$\text{row} = \text{round}\left(-\text{pan}_y + \frac{h_{\text{img}}}{2}\right)$$

데이터 추출:

- 선택된 row의 전체 가로 픽셀 R/G/B 값 추출
- LDR: `pixels[idx] / 255.0f` 정규화
- HDR: `pixels_f32[idx]` 사용, 채널 최대값으로 정규화

렌더링: `ImDrawList::AddPolyline`, 선 두께 1.5f. 픽셀 수가 차트 폭보다 많으면 균등 다운샘플링.

채널	색상 (RGBA)	비고
R	(255, 80, 80, 200)	붉은 라인
G	(80, 255, 80, 200)	녹색 라인
B	(80, 120, 255, 200)	파란 라인

표 28: H-Line Cut 채널 라인 색상

Y축 라벨: 25/50/75/100% 위치에 역정규화 값을 표시.

- LDR: %d 정수 포맷 (예: 255)
- HDR: %.2f 소수점 포맷 (예: 1.23)

X축 틱: `calc_nice_ticks` 알고리즘 — 최대 7구간으로 1/2/5/10 배수 스텝 자동 계산.

라벨 형식: "A: filename.png row=123"

호버 툴팁: 마우스 X 위치에 해당하는 픽셀 인덱스, R/G/B 값 표시. LDR은 %d, HDR은 %.4f 포맷. 3채널 차트 전체에 수직 크로스헤어(흰색 alpha=120) 표시.

8.15. V-Line Cut 창 (Ctrl+Y)

Ctrl+Y 키로 토글하는 `render_vline_cut_window()` 함수가 현재 뷰포트 중앙에 해당하는 수직 라인의 픽셀값을 표시한다. H-Line Cut과 구조가 동일하며 방향만 수직이다.

라인 선택: 뷰포트 `pan_x`를 기반으로 계산.

$$\text{col} = \text{round}\left(-\text{pan}_x + \frac{w_{\text{img}}}{2}\right)$$

데이터 추출: 선택된 col의 전체 세로 픽셀 R/G/B 값 추출 (H-Line Cut과 동일한 HDR/LDR 분기).

라벨 형식: "A: filename.png col=456"

채널 색상, Y축/X축 라벨 형식, 호버 툴팁 모두 H-Line Cut과 동일하다.

8.16. 파일 열기 다이얼로그

av는 SDL3 네이티브 파일 다이얼로그(`SDL_ShowOpenFileDialog`)를 통해 플랫폼 기본 파일 선택 UI를 제공한다.

8.16.1. Open Images 창 (Shift+Cmd+O)

Shift+Cmd+O 키(또는 File 메뉴 → Open Images)로 토글하는 플로팅 창이다.

- **창 폭:** 480px 고정
- **배경 알파:** 0.92f
- **내용:** Image A와 Image B 각각의 파일명 + 해상도 표시 + “Open” 버튼
- **체크박스:** “Clear other image when loading single” — 단일 이미지 로드 시 반대 슬롯 자동 클리어 여부

8.16.2. SDL 파일 다이얼로그 필터

```
SDL_DialogFileFilter filters[] = {
    { "Image files",
      "png;jpg;jpeg;bmp;tga;hdr;ppm;pgm" },
    { "All files", "*" },
};
SDL_ShowOpenFileDialog(callback, userdata, window,
                       filters, 2, nullptr, false);
```

8.16.3. 비동기 처리 흐름

SDL3 파일 다이얼로그는 비동기 콜백으로 결과를 반환한다. `open_dialog_callback()` 이 `open_state.opened_path` 와 `open_pending = true` 를 설정하면, 메인 루프에서 이를 감지하여 이미지를 로딩한다.

8.17. Save 다이얼로그 시스템

`Shift+Cmd+S` 키(또는 File 메뉴 → Save)로 토글하는 컨텍스트 인식 저장 다이얼로그이다. 현재 활성 창에 따라 세 가지 모드로 분기한다.

분기 로직:

- Histogram 활성 → Chart Save (`is_histogram = true`)
- H-Line Cut 또는 V-Line Cut 활성 → Chart Save (`is_histogram = false`)
- Statistics 활성 → Stats Save
- 그 외 → Image Save (기본)

8.17.1. Image Save 다이얼로그

창 폭: 580px, 배경 알파 0.92f

- 포맷 선택: PNG / BMP / PPM (라디오 버튼)
- PPM 옵션: P6(바이너리)/P3(ASCII) + 비트 수 (8/10/12/16)
- 항목: Image A, Image B, Diff 각각 체크박스 + 경로 입력 필드
- 경로 자동 생성: 원본 경로에 `_save` 접미사 + 확장자 자동 변경
- Save All Checked 버튼: 체크된 항목 모두 저장
- 상태 메시지: 성공 시 초록색, 에러 시 빨간색

8.17.2. Chart Save 다이얼로그

창 폭: 610px, 배경 알파 0.92f

- 해상도 설정: 기본 1920×1080 , 최소 320×240
- 채널 모드: Combined (3채널 합산) / Separate (R/G/B 별도 파일)
- PNG 항목: A/B/Diff 각각 체크박스 + 경로
- CSV 항목: A/B/Diff 각각 체크박스 + 경로
- 파일 접미사 자동 결정: Histogram → `_histogram`, H-Line Cut → `_hcut`, V-Line Cut → `_vcut`

8.17.3. Stats Save 다이얼로그

창 폭: 560px, 배경 알파 0.92f

- CSV 전용, A/B/Diff 각각 체크박스 + 경로
- 파일 접미사: `_stats`

8.18. 이미지 회전 UI

`r` 키는 시계방향 90° , `Ctrl+R` 키는 반시계방향 90° 로 모든 로드된 이미지를 동시에 회전한다.

- 처리 범위: 로드된 모든 이미지(`images[0]`, `images[1]`) 동시 적용
- 뷰포트 초기화: 회전 후 모든 뷰포트의 fit-to-window 리셋 (`fit=true`, `pan=0`)
- CPU 기반: GPU 셰이더가 아닌 CPU 픽셀 데이터 직접 변환 후 텍스처 재업로드

8.19. Chart Export 렌더링 파이프라인

차트를 PNG/CSV로 내보내는 기능은 `chart_export.cpp`와 CPU 전용 소프트웨어 렌더러 `SoftRenderer`를 통해 구현된다.

8.19.1. SoftRenderer

`soft_renderer.h` / `soft_renderer.cpp`에 정의된 CPU 기반 RGBA8 캔버스 클래스이다. OpenGL이나 ImGui에 의존하지 않고 독립적으로 이미지를 생성한다.

```
class SoftRenderer {
public:
    SoftRenderer(int width, int height);
    void clear(uint8_t r, uint8_t g, uint8_t b, uint8_t a = 255);
    void fill_rect(int x, int y, int w, int h, uint8_t r, uint8_t g, uint8_t b, uint8_t
a = 255);
    void draw_rect(int x, int y, int w, int h, uint8_t r, uint8_t g, uint8_t b, uint8_t
a = 255);
    void hline(int x0, int x1, int y, uint8_t r, uint8_t g, uint8_t b, uint8_t a =
255);
```

```

void vline(int x, int y0, int y1, uint8_t r, uint8_t g, uint8_t b, uint8_t a =
255);
void draw_polyline(const float* xs, const float* ys, int count,
uint8_t r, uint8_t g, uint8_t b, uint8_t a = 255,
float thickness = 1.5f);
bool load_font(const char* ttf_path, float pixel_height);
void draw_text(int x, int y, const char* text,
uint8_t r, uint8_t g, uint8_t b, uint8_t a = 255);
int text_width(const char* text) const;
bool save_png(const char* path) const;
};

```

- **텍스트:** stb_truetype으로 Roboto-Medium.ttf 14px 래스터화
- **저장:** stbi_write_png() 로 PNG 파일 생성

8.19.2. Histogram PNG 내보내기

export_histogram_png() 함수가 지정 해상도(기본 1920 × 1080)의 PNG를 생성한다.

마진: ml=80, mr=20, mt=40, mb=50 (픽셀)

모드:

- **Combined:** R/G/B 3개 서브차트를 수직 스택, 각 채널 색상 막대
- **Separate:** R/G/B 채널을 각각 별도 파일(_R.png / _G.png / _B.png)로 저장

Y축: 최대 카운트 + “0” (왼쪽 마진, right-align)

X축 틱: {0, 64, 128, 192, 255} 고정 5개

8.19.3. Line Cut PNG 내보내기

export_linecut_png() 함수가 H-Line Cut 또는 V-Line Cut 데이터를 PNG로 내보낸다.

마진: ml=70, mr=20, mt=40, mb=50

렌더링: SoftRenderer::draw_polyline() , 두께 1.5f

채널 색상: R=(255,80,80), G=(80,255,80), B=(80,120,255)

8.19.4. CSV 내보내기

함수	컬럼	행 수
export_histogram_csv	bin,R,G,B	256행
export_linecut_csv	position,R,G,B	n행 (픽셀 수)
export_stats_csv	Metric,R,G,B,Description	13 + 4행

표 29: CSV 내보내기 형식

9. 사용법

이 장에서는 CLI 옵션, 키보드/마우스 단축키, 실전 활용 시나리오를 다룬다.

9.1. CLI 옵션

```
av [options] [imageA] [imageB]
```

옵션	기본값	설명
<code>--diff-mode <mode></code>	<code>none</code>	<code>none abs rel falsecolor ssim</code>
<code>--zoom <factor></code>	<code>fit</code>	초기 줌 레벨 (<code>fit</code> , <code>1</code> , <code>2.0</code> 등)
<code>--sync</code>	<code>on</code>	뷰포트 동기화 활성화
<code>--no-sync</code>	<code>-</code>	뷰포트 동기화 비활성화
<code>--amplify <val></code>	<code>1.0</code>	Diff 강조 배율 (<code>0.1~100.0</code>)
<code>--fullscreen</code>	<code>off</code>	풀스크린으로 시작
<code>--geometry <WxH></code>	<code>1280x720</code>	초기 윈도우 크기
<code>--profile <file></code>	<code>sRGB</code>	ICC 색 프로파일 경로
<code>--no-color-mgmt</code>	<code>off</code>	색 관리 비활성화
<code>-p</code> , <code>--pan-step <N></code>	<code>32</code>	Shift+hjkl 이동 크기 (픽셀)
<code>-bc <A> <D></code>	<code>mag/yel/</code> <code>cya</code>	패널 경계 색상 (6자리 hex)
<code>-d</code> , <code>--diff</code>	<code>-</code>	<code>--diff-mode abs</code> 단축
<code>--windowed</code>	<code>off</code>	윈도우 모드로 시작 (타이틀바 + 리사이즈)
<code>-nb</code>	<code>off</code>	패널 테두리 숨김 상태로 시작
<code>--version</code>	<code>-</code>	버전 출력 후 종료
<code>-h</code> , <code>--help</code>	<code>-</code>	도움말 출력

표 30: CLI 옵션 전체 목록

위치 인자(positional arguments)로 `imageA` 와 `imageB` 를 순서대로 지정한다. 하나만 지정하면 단일 이미지 모드로 시작한다.

9.2. 키보드 단축키

9.2.1. 줌 조작

키	동작
<code>+</code> / <code>=</code> / Keypad <code>+</code>	확대 (다음 2^n 줌 레벨)
<code>-</code> / Keypad <code>-</code>	축소 (이전 2^n 줌 레벨)
<code>Z</code>	확대 / <code>Shift+Z</code> : 축소
<code>X</code>	축소 (<code>Z</code> / <code>X</code> 쌍으로 줌 인/아웃)
<code>0 ~ 8</code>	2^n 배 줌 직접 설정 ($0 \rightarrow 1\times$, $1 \rightarrow 2\times$, ..., $8 \rightarrow 256\times$)
<code>Space</code>	1:1 ($1.0\times$) 줌 + 중앙 이동
<code>F</code>	Fit-to-Window 토글

표 31: 줌 조작 단축키

사용 가능한 줌 레벨: `0.125`, `0.25`, `0.5`, `1.0`, `2.0`, `4.0`, `8.0`, `16.0`, `32.0`, `64.0`, `128.0`, `256.0` (총 12단계)

9.2.2. 패닝 / 내비게이션

키	동작
<code>H</code> / <code>←</code>	좌측 패닝 (1px)
<code>L</code> / <code>→</code>	우측 패닝 (1px)
<code>K</code> / <code>↑</code>	위 패닝 (1px)
<code>J</code> / <code>↓</code>	아래 패닝 (1px)
<code>Shift+H/L/K/J</code>	빠른 패닝 (<code>pan_step</code> 픽셀, 기본 32px)
<code>Cmd+Shift+H/L/K/J</code>	각 방향 끝단으로 점프
<code>G</code>	뷰포트 중앙으로 이동

표 32: 패닝/내비게이션 단축키

9.2.3. 비교 모드 전환

키	동작
Ctrl+3	Diff: Pixel Absolute
Ctrl+4	Diff: Pixel Relative
Ctrl+5	Diff: FalseColor
Ctrl+6	SSIM Map (비동기 계산)
Ctrl+D	Diff 비활성화 (mode=None)
S	뷰포트 동기화 토클
Tab	A/B 패널 포커스 전환 (0↔1)
Shift+Space	A↔B 이미지 스왑 (양쪽 로드 시에만)

표 33: 비교 모드 전환 단축키

9.2.4. Diff 파라미터

키	동작
[	Diff amplify - 0.5 (최소 0.5×)
]	Diff amplify +0.5 (최대 100×)
\u{005C}	Diff amplify 1.0×으로 초기화

표 34: Diff 파라미터 조작 단축키

9.2.5. 채널 분리

키	동작
Shift+R	Red 채널만 표시
Shift+G	Green 채널만 표시
Shift+B	Blue 채널만 표시
Shift+C	RGB 전체 컬러 (기본)

표 35: 채널 분리 단축키

9.2.6. UI 토글

키	동작	비고
U	메뉴바 / 상태바 토글	
I	이미지 정보 팝업 토글	
P	Pathfinder Image 미니맵 토글 (모드 0↔1)	
Ctrl+P	Pathfinder Schematic 미니맵 토글 (모드 0↔2)	
V	픽셀 정보 풍선말 토글	
R	이미지 시계방향 90° 회전	모든 로드 이미지 동시
Ctrl+R	이미지 반시계방향 90° 회전	모든 로드 이미지 동시
Shift+Cmd+O	네이티브 파일 다이얼로그 열기 (활성 패널)	
Shift+Cmd+S	Save 다이얼로그 토글	컨텍스트 인식
Ctrl+H	Histogram 창 토글	상호 배타적
Ctrl+L	H-Line Cut 창 토글	상호 배타적
Ctrl+Y	V-Line Cut 창 토글	상호 배타적
Ctrl+S	Statistics 창 토글	상호 배타적
Ctrl+E	ROI 선택 모드 토글	ESC로 해제
O	Overlay/Blend 비교 모드 토글	ESC로 해제
Ctrl+T	Scatter Plot 창 토글	
W	윈도우 모드 토글 (타이틀바 + 리사이즈)	--windowed 와 동일
B	패널 테두리 토글	-nb 로 초기 off 가능; av.ini 영속화
Ctrl+X	픽셀값 형식 순환 (Dec → 0xHex → Hexh)	av.ini 영속화
Ctrl+Shift+H	핫키 도움말 창 토글	
Q	애플리케이션 종료	

표 36: UI 토글 단축키

9.2.7. 시퀀스 탐색

키	동작
N	활성 패널의 다음 시퀀스 프레임 로드
Shift+N	활성 패널의 이전 시퀀스 프레임 로드

표 37: 시퀀스 탐색 단축키

9.3. 마우스 조작

동작	효과
좌클릭 드래그	이미지 패닝
스크롤 휠	줌 인/아웃
우클릭 드래그	영역 선택 줌 (drag-to-zoom)
파일 드롭 (첫 번째)	Image A로 로드
파일 드롭 (두 번째)	Image B로 로드

표 38: 마우스 조작 일람

9.4. 사용 예시

```
# 단일 이미지 열기
av image.png

# 두 이미지 side-by-side 비교
av before.png after.png

# Pixel Absolute diff, 5배 강조
av --diff-mode=abs --amplify=5 ref.png output.png

# SSIM 분석 모드로 시작
av --diff-mode=ssim render_A.png render_B.png

# Display P3 ICC 프로파일 적용
av --profile=/path/to/DisplayP3.icc a.png b.png

# 2x 줌, 동기화 비활성화, 커스텀 경계 색상
av --zoom=2 --no-sync -bc ff0000 0000ff 00ff00 a.jpg b.jpg
```

9.5. 워크플로우 시나리오

9.5.1. 시나리오 1: 렌더링 회귀 테스트

1. `av --diff-mode=abs --amplify=10 golden.png current.png` 실행
2. FalseColor 모드로 전환 (`Ctrl+5`)
3. `] 키`로 amplify 증가하여 미세 차이 확인
4. `Ctrl+S` 로 Statistics 창 열어 MSE/PSNR 수치 확인

9.5.2. 시나리오 2: 색 보정 검토

1. `av --profile=DisplayP3.icc before_cc.png after_cc.png` 실행
2. `Shift+R`, `Shift+G`, `Shift+B` 키로 채널별 확인
3. `s` 키로 동기화 활성화 후 양쪽 패널 동시 패닝
4. 줌 인하여 특정 영역의 색 차이를 세밀히 비교

9.5.3. 시나리오 3: 머신러닝 출력 분석

1. `av --diff-mode=ssim gt.png predicted.png` 실행
2. SSIM 히트맵 로딩 완료 대기 (상태바에 `SSIM: computing...` 표시)
3. 상태바에서 SSIM 점수 확인 (≥ 0.99 : 초록, ≥ 0.90 : 노랑, < 0.90 : 빨강)
4. `+` 키 반복으로 고배율 줌 진입 후 세부 차이 분석

9.5.4. 시나리오 4: ROI 영역 집중 분석

1. `av ref.png output.png` 실행
2. `Ctrl+E` 로 ROI 선택 모드 활성화 (상태바에 `[ROI]` 표시)
3. 비교할 영역을 마우스 드래그로 선택 (노란 점선 사각형 오버레이 표시)
4. ROI Stats 창에서 A/B/Diff의 Mean, Std, Min, Max 수치 비교
5. `Ctrl+S` 로 전체 Statistics와 비교하여 문제 영역을 좁혀 확인

9.5.5. 시나리오 5: 비디오 프레임 시퀀스 비교

1. `av frame_0001.png` 실행 (동일 디렉토리에 다수 프레임 존재)
2. 상태바 `[1/120]` 형식으로 전체 프레임 수 확인
3. `N` 키로 다음 프레임, `Shift+N` 으로 이전 프레임 탐색
4. `0` 키로 Overlay 모드 활성화 후 인접 프레임 블렌드 비교
5. View 메뉴 Overlay 슬라이더로 alpha 조정하여 전환 분석

9.5.6. 시나리오 6: Scatter Plot 색 분포 분석

1. `av original.png processed.png` 실행

2. **Ctrl+T** 로 Scatter Plot 창 열기
3. R/G/B 채널 각각의 A vs B 픽셀 분포 확인
4. 점군이 $y=x$ 대각선에서 벗어난 정도로 색 왜곡 정도 파악
5. ROI 선택 후 특정 영역의 색 분포만 추출하여 분석

10. 앱 아이콘

Advanced Pixel Lens는 128×128 RGBA PNG 아이콘을 소스코드 내에서 프로시저럴하게 생성한다. 외부 이미지 에셋에 의존하지 않고, 코드만으로 아이콘을 래스터화하여 빌드 시점에 포함한다.

10.1. 프로시저럴 생성

main.cpp의 `create_app_icon()` 함수가 아이콘을 생성한다. 128×128 크기의 RGBA8 픽셀 배열을 직접 그린 뒤 SDL 윈도우 아이콘으로 설정한다.

```
// 128×128 RGBA8 픽셀 데이터를 직접 래스터화
std::vector<uint32_t> pixels(128 * 128, 0);
// ... 각 구성 요소를 픽셀 단위로 그린 ...
SDL_Surface* icon = SDL_CreateRGBSurfaceFrom(
    pixels.data(), 128, 128, 32, 128*4,
    0x000000FF, 0x0000FF00, 0x00FF0000, 0xFF000000);
SDL_SetWindowIcon(window, icon);
```

10.2. 128×128 디자인 구조

아이콘은 다음 네 가지 주요 요소로 구성된다.

요소	색상	세부 사항
배경	Dark Navy	#1a1f2e (RGB 26,31,46), 라운드 14px
3×3 컬러 그리드	9색	마진 10px, 갭 2px, 셀 ~34px, alpha=230
돋보기 렌즈	흰색	중심 (80,80), 반지름 28px, 링 3px, alpha=210
돋보기 손잡이	흰색	45° 대각선, 길이 12px

표 39: 앱 아이콘 구성 요소

10.2.1. 3×3 컬러 그리드

이미지 비교기의 정체성을 표현하는 9개 색상 셀:

열 1	열 2	열 3
Magenta (255,60,200)	Cyan (40,220,255)	Yellow (255,220,30)
Red (255,70,70)	Green (70,220,90)	Blue (70,120,255)
Orange (255,155,30)	Mint (30,220,175)	Violet (190,70,255)

표 40: 3×3 컬러 그리드 색상 배치

10.2.2. 돋보기 오버레이

그리드 우하단에 돋보기(magnifier) 아이콘을 겹쳐 그려 “픽셀을 자세히 들여다본다”는 앱의 목적을 상징한다.

- **렌즈 링:** 중심 (80,80), 반지름 28px, 흰색 3px 링 (`alpha=210`)
- **손잡이:** 렌즈 에지에서 45° 대각선 방향으로 12px 돌출
- **렌즈 내부:** 배경 그리드가 투과되어 보임

11. Phase 5: 분석 기능 확장

v0.3에서 네 가지 고급 분석 기능이 추가되었다: ROI 통계, 이미지 시퀀스 탐색, Overlay/Blend 비교 모드, Scatter Plot 시각화.

11.1. ROI (Region of Interest) 선택 + 영역 통계

ROI 모드는 `Ctrl+E` 로 활성화되며, 이미지 패널에서 마우스 드래그로 관심 영역을 선택한 뒤 해당 영역의 통계를 별도 창에 표시한다.

11.1.1. ROI 드래그 선택 흐름

```
Ctrl+E → roi.active = true
좌클릭 드래그 → handle_roi_drag():
    drag_sx/sy = 시작 화면 좌표
    매 프레임: x/y/w/h = screen_to_image(drag_sx,sy, cur_sx,sy)
    mouse up → has_roi = true
ESC → roi.active = false, has_roi = false
```

드래그 시 이미지 좌표계로 변환하기 위해 `screen_to_image()` 변환을 적용한다. 뷰포트의 줌·팬 상태에 관계없이 항상 이미지 픽셀 좌표로 ROI가 저장된다.

11.1.2. ROI 오버레이 렌더링

선택된 ROI는 ImGui `DrawList` 를 사용하여 노란 점선 사각형 (`ImVec4(1,1,0,0.8)`)으로 패널 위에 오버레이 렌더링된다. ROI 모드가 활성화된 동안 상태바에 `[ROI]` 레이블과 좌표/크기가 표시된다.

11.1.3. compute_roi_stats

```
ImageStats compute_roi_stats(const ImageEntry& img,
                             int rx, int ry, int rw, int rh);

ImageStats compute_roi_diff_stats(const ImageEntry& a, const ImageEntry& b,
                                  int rx, int ry, int rw, int rh,
                                  DiffExtraStats& extra);
```

`chart_export.cpp` 에 구현되어 있으며, 지정 영역의 픽셀을 순회하여 Mean, Std, Min, Max를 채널 별로 계산한다. `render_roi_stats_window()` 가 A/B/Diff 3열 테이블로 결과를 표시한다.

11.1.4. 상태바 표시

ROI 활성 시 상태바는 다음 정보를 표시한다:

항목	예시
ROI 모드 레이블	[ROI]
ROI 좌표+크기	ROI: (120, 80) 200x150

표 41: ROI 활성 시 상태바 표시

11.2. 이미지 시퀀스 탐색

이미지 파일을 열면 `scan_image_directory()` 가 동일 디렉토리 내 이미지 파일 목록을 자동 스캔하여 `SequenceState.files` 에 저장한다. 이후 `N` / `Shift+N` 키로 시퀀스를 탐색할 수 있다.

11.2.1. scan_image_directory

```
std::vector<std::string> scan_image_directory(
    const std::string& current_path,
    int& current_out); // 현재 파일의 인덱스를 출력
```

지원 확장자(`.png`, `.jpg`, `.jpeg`, `.bmp`, `.tga`, `.pgm`, `.ppm`, `.hdr`)에 해당하는 파일을 열거하고 natural sort (파일명의 숫자 부분을 수치로 비교)로 정렬한다. 이로써 `frame_002.png` 이 `frame_10.png` 보다 앞에 오는 문제가 해결된다.

11.2.2. 탐색 키 처리

`N` / `Shift+N` 키 처리는 `app.cpp` 의 `handle_keyboard()` 에 구현되어 있다:

```
case SDL_SCANCODE_N: {
    auto& seq = state.sequences[ap];
    if (!seq.files.empty() && seq.current_index >= 0) {
        int n = (int)seq.files.size();
        if (shift) seq.current_index = (seq.current_index - 1 + n) % n;
        else      seq.current_index = (seq.current_index + 1) % n;
        state.open_state.opened_path = seq.files[seq.current_index];
        state.open_state.open_target = ap;
        state.open_state.open_pending = true;
        state.open_state.clear_other = false;
    }
}
```

`open_pending` 플래그를 통해 기존 이미지 로딩 파이프라인을 재사용하므로, 캐싱과 색 관리가 자동으로 적용된다.

11.2.3. 상태바 시퀀스 표시

시퀀스가 스캔되면 상태바의 패널 레이블에 현재/전체 인덱스가 추가된다:

상황	표시 예시
시퀀스 없음	A: frame_001.png
시퀀스 3번째/24개	A [3/24]: frame_003.png

표 42: 상태바 시퀀스 인덱스 표시

11.3. Overlay/Blend 비교 모드

0 키로 활성화되며, 두 이미지 패널 대신 단일 블렌드 패널 하나를 렌더링한다. 두 가지 서브 모드를 지원한다.

11.3.1. Blend 모드

Alpha 가중치 `overlay.alpha ∈ [0, 1]` 로 A/B 텍스처를 혼합한다:

```
// BLEND_FRAG_SRC (blend + curtain dual-mode shader)
uniform float u_alpha;
uniform int    u_mode;          // 0=Blend, 1=Curtain
uniform float u_curtain_x;     // Curtain 구분선 위치 [0,1]

vec4 colorA = texture(u_texA, v_uv);
vec4 colorB = texture(u_texB, v_uv);
if (u_mode == 0) {
    out_color = mix(colorA, colorB, u_alpha);
} else {
    out_color = (v_uv.x < u_curtain_x) ? colorA : colorB;
}
```

View 메뉴에 alpha 슬라이더(0→1)와 Blend/Curtain 라디오 버튼이 제공된다.

11.3.2. Curtain 모드

화면을 수직으로 분할하여 좌측(`u_uv.x < curtain_x`)은 A, 우측은 B를 표시한다. `curtain_x`는 마우스 드래그로 조정 가능하다.

11.3.3. 레이아웃 전환

Overlay 활성 시 `main_window.cpp`의 레이아웃 로직이 2-panel 배치 대신 단일 `ImagePanel`을 전체 너비로 렌더링한다. 비활성화하면 원래 side-by-side 레이아웃으로 돌아온다.

11.3.4. 상태바 표시

Overlay 활성 시 상태바에 현재 모드와 alpha 비율이 표시된다:

모드	표시 예시
Blend 모드	[Overlay: Blend 50%]
Curtain 모드	[Overlay: Curtain]

표 43: Overlay 모드 상태바 표시

11.4. Scatter Plot

`Ctrl+T`로 토글하는 Scatter Plot 창은 A/B 이미지의 대응 픽셀 값 분포를 채널별로 시각화하여 색 왜곡, 감마 차이, 채도 변화 등을 직관적으로 파악할 수 있게 한다.

11.4.1. `extract_scatter_plot`

```
struct ScatterPlotData {
    std::vector<float> r_a, r_b; // R channel 샘플
    std::vector<float> g_a, g_b; // G channel 샘플
    std::vector<float> b_a, b_b; // B channel 샘플
    int total_pixels = 0;
    int sampled      = 0;
    bool valid       = false;
};

ScatterPlotData extract_scatter_plot(const ImageEntry& a,
                                     const ImageEntry& b,
                                     int max_samples = 20000);
```

이미지 전체 픽셀을 `max_samples` 개수로 랜덤 샘플링하여 정규화된 `[0,1]` 범위의 (A값, B값) 쌍을 추출한다. A와 B의 크기가 다를 경우 작은 이미지 크기 기준으로 처리한다.

11.4.2. 시각화

`render_scatter_plot_window()`는 `ImDrawList`를 사용하여 R/G/B 채널별 산점도를 그린다:

요소	설명
그리드	0.25 간격 밝은 회색 격자선
$y=x$ 기준선	점군이 완벽히 일치할 때 놓이는 대각선 (회색)
R 채널 점	빨간색 (<code>ImVec4(1, 0.3, 0.3, 0.5)</code>)
G 채널 점	초록색 (<code>ImVec4(0.3, 1, 0.3, 0.5)</code>)
B 채널 점	파란색 (<code>ImVec4(0.3, 0.5, 1, 0.5)</code>)
축 레이블	$X = A$ 값, $Y = B$ 값, 범위 $[0,1]$
샘플 수	창 하단에 <code>n=20000 / 1,048,576 pixels</code> 표시

표 44: Scatter Plot 시각화 요소

점군이 $y=x$ 대각선에 밀집할수록 두 이미지가 동일하다는 것을 의미한다. 기울기나 곡선 패턴은 감마 보정이나 톤 매핑 차이를 나타낸다.

11.5. Linux AppImage 아키텍처 자동 감지

v0.3에서 `CMakeLists.txt` 의 AppImage 빌드 타겟이 호스트 아키텍처를 자동으로 감지하여 적합한 `linuxdeploy` 바이너리를 다운로드한다.

```
execute_process(COMMAND uname -m OUTPUT_VARIABLE HOST_ARCH ...)
if(HOST_ARCH STREQUAL "aarch64" OR HOST_ARCH STREQUAL "arm64")
    set(LD_ARCH "aarch64")
else()
    set(LD_ARCH "x86_64")
endif()
set(LINUXDEPLOY "${APPIMAGE_DIR}/linuxdeploy-${LD_ARCH}.AppImage")
set(APPIMAGE_OUT "${CMAKE_SOURCE_DIR}/bin/av-${LD_ARCH}.AppImage")
```

출력 파일은 `bin/av-aarch64.AppImage` 또는 `bin/av-x86_64.AppImage` 로 생성된다.

12. Phase 6: v0.11 기능 확장

v0.11에서 다섯 가지 신규 분석 기능과 Dynamic Window Title이 추가되었다.

12.1. Crosshair Overlay

M 키로 토글하는 십자선(Crosshair) 오버레이이다.

12.1.1. 상태 필드

`AppState::show_crosshair` (bool) — 활성 시 커서 위치에 수평/수직 점선을 그린다.

12.1.2. 렌더링 방식

`ImagePanel::render()` 내부에서 `ImDrawList` 를 사용하여 구현한다. 커서가 패널 영역 내에 있을 때, 커서의 x 좌표로 수직선, y 좌표로 수평선을 패널 전체 너비/높이에 걸쳐 그린다. 반투명 노란색 (`ImVec4(1, 1, 0, 0.6)`) 1px 선을 사용한다.

12.1.3. 키 바인딩

키	동작
<code>M</code>	Crosshair 오버레이 토글
<code>Escape</code>	Crosshair 비활성화 (다른 팝업 해제 후)

표 45: Crosshair 키 바인딩

12.2. PSNR 자동 계산

두 이미지가 로드되고 diff 모드가 활성화되면 PSNR(Peak Signal-to-Noise Ratio)을 자동으로 계산하여 상태바에 표시한다.

12.2.1. 구현

`DiffState` 에 `psnr_db` (float)와 `psnr_computed` (bool) 캐시 필드가 있다. `compute_diff_stats()` 함수를 재사용하여 이미지 변경 시 한 번만 계산한다. 상태바에 `PSNR: %.2f dB` 형식으로 표시되며, 무한대(동일 이미지)일 경우 `PSNR: inf` 로 표시한다.

12.2.2. 캐시 무효화

이미지 로드, diff 모드 변경, 이미지 회전 시 `psnr_computed = false` 로 리셋되어 다음 프레임에서 재계산된다.

12.3. Tolerance Diff

diff 모드에서 threshold 기반으로 유의미한 차이가 있는 픽셀만 하이라이트하는 기능이다.

12.3.1. 상태 필드

- `DiffState::threshold` (int, 0~255) — 0이면 비활성, 양수면 해당 값 초과 차이만 표시
- `DiffState::threshold_exceed_count` (int) — threshold 초과 픽셀 수 (캐시)
- `DiffState::threshold_total_count` (int) — 비교된 전체 픽셀 수

12.3.2. GPU/CPU 이중 경로

OpenGL 모드에서는 셰이더 uniform `u_threshold` 로 전달되어 GPU에서 처리한다. 소프트웨어 렌더러에서는 CPU 루프에서 동일한 로직을 수행한다. 두 경로 모두 threshold 이하의 차이는 검정색 (0,0,0)으로, 초과 시에만 증폭된 diff 값을 표시한다.

12.3.3. 키 바인딩

키	동작
<code>Ctrl+7</code>	Tolerance diff 토글 (threshold 0↔1)
<code>Shift+] </code>	Threshold +1
<code>Shift+[</code>	Threshold -1
<code>Ctrl+\\</code>	Threshold 리셋 (0)

표 46: Tolerance Diff 키 바인딩

12.4. Slideshow 자동 재생

시퀀스 내 이미지를 자동으로 순환하는 기능이다.

12.4.1. SlideshowState

```
struct SlideshowState {
    bool active = false;
    float interval = 3.0f; // seconds
    float countdown = 0.0f;
    int panel = 0; // which panel's sequence to advance
};
```

12.4.2. DeltaTime 기반 카운트다운

`main_window.cpp`의 `render` 루프에서 `ImGui`의 `io.DeltaTime`을 사용하여 카운트다운을 감소시킨다. 카운트다운이 0 이하가 되면 해당 패널의 시퀀스에서 다음 이미지를 로드하고 카운트다운을 `interval`로 리셋한다.

12.4.3. 키 바인딩

키	동작
A	Slideshow 토글
Shift+↑	간격 +1초 (최대 10초)
Shift+↓	간격 -1초 (최소 0.5초)

표 47: Slideshow 키 바인딩

12.5. Histogram Compare

히스토그램 창에서 A/B 이미지의 히스토그램을 동시에 오버레이하여 비교하는 모드이다.

12.5.1. compare 모드 체크박스

히스토그램 창 상단에 “Compare” 체크박스를 제공한다. 체크 시 `AppState::histogram_compare` (bool)가 토글되어 두 이미지의 히스토그램이 동일 좌표계에 반투명으로 겹쳐 표시된다.

12.5.2. 이중 히스토그램 오버레이

단일 패널에 A 이미지의 히스토그램(실선)과 B 이미지의 히스토그램(점선 또는 반투명)을 겹쳐 그린다. 각 채널(R/G/B)별로 두 이미지의 분포 차이를 직관적으로 비교할 수 있다.

12.6. Dynamic Window Title

Windowed 모드에서 SDL 창 제목에 로드된 이미지 파일명과 활성 diff 모드를 표시한다.

12.6.1. 타이틀 포맷

상태	제목
이미지 없음	av
A만 로드	av - photo.png
A+B 로드	av - photo.png ref.png
A+B + diff	av - photo.png ref.png [Abs]
Fullscreen	Advanced Pixel Lens (기존 유지)

표 48: Dynamic Window Title 포맷

12.6.2. 구현

`main_window.cpp` 의 `MainWindow::render()` 초반부에서 매 프레임 타이틀 문자열을 구성한다. 이전 프레임의 타이틀과 비교하여 변경이 있을 때만 `SDL_SetWindowTitle()` 을 호출하여 불필요한 API 호출을 방지한다.

`title_diff_label()` 헬퍼 함수가 `DiffState::Mode` 를 짧은 라벨 문자열로 변환한다: `PixelAbsolute` → "Abs", `PixelRelative` → "Rel", `FalseColor` → "FalseColor", `SSIM` → "SSIM". `diff` 모드가 `None` 이면 라벨을 생략한다.

12.7. Border Toggle, Pixel Format 및 av.ini 설정 영속화

패널 테두리와 픽셀값 표시 형식을 런타임에 토글하고 설정을 `av.ini` 파일에 영속화하는 기능이다.

12.7.1. 토글 메커니즘

B 키(단독, modifier 없음)를 누르면 `state.show_borders` 가 반전된다. `Shift+B` 는 기존의 Blue 채널 모드 전환으로 유지된다. View 메뉴에서도 “Show Borders” 항목으로 동일한 토글이 가능하다.

토글이 `false` 일 때 비활성화되는 렌더링:

- `image_panel.cpp` 의 7개 `draw_image_border()` 호출 (이미지 경계)
- `main_window.cpp` 의 포그라운드 패널 보더 (그림자 + 컬러 경계 사각형)

`Ctrl+X` 를 누르면 `state.pixel_format` 이 `Decimal` → `Hex0x` → `HexH` → `Decimal` 로 순환한다. X 키 단독은 기존 줌 아웃 기능을 유지한다. View 메뉴의 “Pixel Format” 서브메뉴에서 개별 형식을 직접 선택할 수도 있다.

12.7.2. av.ini 영속화

`av.ini` 는 작업 디렉토리에 생성되는 간단한 `key=value` 파일이다.

```
# av configuration
show_borders=1
pixel_format=0
```

`pixel_format` 값: 0=Decimal, 1=Hex0x, 2=HexH.

로드 순서: INI → CLI 적용. `-nb` CLI 옵션은 INI의 `show_borders=1` 을 override하여 `false` 로 설정한다.

- `load_app_ini()` : `main.cpp` 에서 `AppState` 생성 직후, `apply_cli_options()` 호출 전에 실행
- `save_app_ini()` : 메인 루프 종료 후, 정리 전에 실행

Advanced Pixel Lens – 구현 가이드 v0.13

2026-03-16

이 문서는 av 이미지 뷰어/비교기의 완전한 구현 참조 문서입니다.